SCO 310: COMPONENT PROGRAMMING — PROJECT REPORT

A Component-Oriented QR Code Generator built from interchangeable JavaBeans components

Unit	SCO 310 / SIT 300 — Component Programming
Project title	`qr-component-suite` — a QR Code generator assembled from JavaBeans components
Student name	`[STUDENT NAME]`
Registration number	`[REGISTRATION NUMBER]`
Course lecturer	Dr. Bernard Ondara, Ph.D.
Institution	Kenyatta University — Department of Computing
Submission date	`[DATE]`
Source tree	`qr-component-suite/` (48 Java source files, 6 581 lines; 51 compiled classes)
Automated verification	108 checks — **108 passed, 0 failed** (transcript: `docs/evidence/test-report.txt`)

Contents

Chapter	Page of the argument
1. Executive summary	what was built, and the headline results
2. Scope, objectives and deliverables	what the project had to achieve, and where each deliverable lives
3. Component-oriented analysis	the COP requirements, the decomposition, the five component characteristics, COP versus OOP, and the three kinds of interface incompatibility
4. The JavaBeans component model	the eight bean steps, the four property kinds, the connection and deployment models, persistence, introspection
5. The component catalogue, component by component	provided/required interfaces, hidden behaviour, properties, events and contract checks for all eleven entries
6. Assembling the components	the composition pipeline, two front ends from one component set, replaceability, persistence of the assembly
7. The encoding logic, in detail	ISO/IEC 18004 stage by stage, with worked examples whose codewords match the standard
8. Guide to the screenshots, listings and figures	which screenshot proves what
9. Verification	strategy, results, the seven defects the evidence caught, negative controls, known divergences
10. Building, running and submitting	commands, marking-criteria mapping, conclusion, references
Appendices	evidence index, glossary, and the course questions answered from this project

1. Executive summary

This project delivers a **working QR Code generator** that is built the way component programming requires: the application is *assembled from independent, replaceable, introspectable components* rather than written as one monolithic program.

The suite contains **nine components plus two infrastructure classes**, every one of them a JavaBean that satisfies the mandatory bean characteristics (public class, no-argument constructor, `Serializable`, `get`/`set` design patterns, delegation

events). The components are wired by a component registry, configured by a deployment descriptor, and consumed by **two different front-end assemblies** — a Swing desktop application and an HTTP web service — both built from the *same* compiled component set.

The QR encoding itself (numeric, alphanumeric and byte modes, Reed–Solomon error correction over GF(2^8), all eight mask patterns, versions 1–40) is implemented from **ISO/IEC 18004 without any third-party dependency**, and a second encoder component wraps the third-party ZXing engine behind an *identical provided interface* to demonstrate replaceability and interface standardisation.

Every claim in this report is backed by machine-generated evidence: the automated suite decodes each produced symbol with the **independent ZXing decoder**, compares the ISO/IEC 18004 parameter tables with the tables published by ZXing (160 version/level combinations, **0 mismatches**), reconstructs each rendered artifact module by module, and runs negative controls that prove the checks can fail — including a corrupted-artifact control and the historical defect that the suite was designed to catch.

Headline results

Result	Value	Evidence
Automated checks	108 total, 108 passed, 0 failed	`docs/evidence/test-results.csv`
ISO/IEC 18004 table cross-check	160/160 combinations identical to ZXing	`docs/evidence/table-cross-check.csv`
Round-trip decode, payload × level	60/60 payloads recovered exactly (incl. accents, CJK, emoji)	`docs/evidence/test-report.txt`
Capacity sweep	160/160 version/level pairs decoded at exact byte capacity	`docs/evidence/capacity-matrix.csv`
Renderer reconstruction	PNG pixels, SVG rectangles and ASCII glyphs rebuild the matrix exactly	`docs/evidence/renderer-matrix.csv`
Artifact gallery	28 artifacts (20 raster, 4 SVG, 4 ASCII) with SHA-256 digests and recovered payloads	`docs/evidence/artifact-gallery.csv`
Code size	6 581 lines of Java across 48 files, 0 external dependency for the engine	`ProjectBuilder` output

---

2. Scope, objectives and deliverables

2.1 Scope

A QR Code generator that

1. accepts arbitrary text (Latin, accented, CJK, emoji, multi-line, vCard and Wi-Fi payloads);
2. produces ISO/IEC 18004 conformant symbols at every version 1–40 and every error-correction level L/M/Q/H;
3. renders the symbol as a PNG raster image, a vector SVG document or console text art;
4. persists both the artifacts and a generation history;
5. is usable from a desktop GUI and an HTTP endpoint;
6. is decomposed into components with explicit provided/required interfaces and can be reconfigured *without recompiling*.

2.2 Objectives (mapped to the marking criteria of the unit)

Objective	How it is achieved	Where to look
Apply component-oriented design, not just object-oriented code	component identification, roles, provided/required interfaces, black-box internals	§4, `docs/diagrams/figure-1-component-model.png`
Implement a real JavaBeans component model	four property kinds, delegation events, introspection, BeanInfo, persistence	§5, `ke.ku.sco310.qr.tools.BeanInspector`
Demonstrate reusability and replaceability	two encoder components, three renderer components, two assemblies	§6, `docs/screenshots/03-gui-png-assembly.png`
Demonstrate the deployment model	JAR with `Java-Bean: True` manifest entries, XML assembly persistence	§5.4, `docs/screenshots/14-jar-manifest.png`
Produce correct, verified logic	108 automated checks against an independent oracle	§9, `docs/evidence/`
Produce complete documentation	this report, screenshots of code and of the running programs	§8, `docs/screenshots/`

2.3 Deliverables inside the submission folder

```
qr-component-studio/
├── src/main/java/ke/ku/sco310/qr/     48 Java sources: components, engine, front ends, build tool
├── src/test/java/ke/ku/sco310/qr/test/ verification suite (8 files)
├── config/assembly.xml                 deployment descriptor (component selection + properties)
├── manifest/MANIFEST.MF                JAR manifest with Java-Bean entries
├── lib/                                optional third-party component (ZXing 3.5.3)
├── pom.xml, README.md                  optional Maven build, project readme
├── QrStudio.jar                        standalone desktop application (double-click / java -jar)
├── out/                                compiled classes, distribution JAR, generated artifacts
└── docs/
    ├── Documentation.md                this report (master document)
    ├── SCO310-QR-Component-Suite-Report.html / .docx / .pdf
    ├── diagrams/                       six generated figures
    ├── screenshots/                    22 screenshots (code, GUI, web service, introspection)
    ├── evidence/                       verification transcript, CSVs, artifact gallery
    └── report-tooling/                 helpers that produced the figures and screenshots
```

---

3. Component-oriented analysis

3.1 What component-oriented programming requires

COP builds systems by **assembling pre-existing, reusable and independent components**. A component is a deployable unit of software with a well-defined interface whose implementation is hidden; the classic formal definition used in the course material (Szyperski, 2002) is:

> A software component is a unit of composition with contractually specified interfaces and explicit context dependencies only. A component can be deployed independently and is subject to third-party composition.

The course notes reduce this to the triple **C = (I, D, B)**: the interfaces a component *offers* (I), the interfaces it *requires* (D), and its *hidden* behaviour (B). Section 4.2 lists the triple for every component of this suite; the hidden behaviour (*B*) lives exclusively inside the `ke.ku.sco310.qr.engine` package, which is deliberately package-private so that no consumer can depend on it.

3.2 Component identification and decomposition

Decomposition followed the *separation of concerns* principle of COP: encoding (pure computation), presentation (image generation), persistence (storage), notification (logging/statistics) and orchestration were separated into different components because they change for different reasons and at different rates.

#	Role	Component (componentId)	Provides (I)	Requires (D)	Hidden behaviour (B)
1	encoder	`PureJavaQrEncoderBean` (`encoder.pure-java.v1`)	`QrCodeEncoder`, `BoundPropertySource`, `Serializable`	—	the whole ISO/IEC 18004 engine (segment analysis, version choice, Reed–Solomon, masking)
2	encoder	`ZxingQrEncoderAdapterBean` (`encoder.zxing-adapter.v1`)	`QrCodeEncoder`, `BoundPropertySource`, `Serializable`	`com.google.zxing.core` (optional library)	translation of the foreign `ByteMatrix` result into the suite's `QrCode` value object
3	renderer	`PngRendererBean` (`renderer.png.v1`)	`QrCodeRenderer`, `BoundPropertySource`	—	raster geometry, run-length drawing, `ImageIO` encoding
4	renderer	`SvgRendererBean` (`renderer.svg.v1`)	`QrCodeRenderer`, `BoundPropertySource`	—	SVG document model, horizontal run merging, millimetre sizing
5	renderer	`AsciiRendererBean` (`renderer.ascii.v1`)	`QrCodeRenderer`, `BoundPropertySource`	—	glyph mapping and aspect-ratio correction
6	repository	`FileSystemRepositoryBean` (`repository.file-system.v1`)	`QrCodeRepository`, `BoundPropertySource`	file system	artifact writing, history serialization, file-name sanitising
7	history	`QrHistoryBean` (`history.bounded-memory.v1`)	indexed/bound/constrained properties	—	capacity policy, overflow accounting
8	listener	`AuditLogListenerBean` (`listener.audit-log.v1`)	`QrCodeGenerationListener`	`QrCodeGeneratedEvent`	log line formatting, file appending
9	facade	`QrStudioFacadeBean` (`facade.qr-studio.v1`)	`generate(...)`, `renderWith(...)`, listener registration	`QrCodeEncoder`, `QrCodeRenderer`, `QrCodeRepository`, `QrHistoryBean`	generation workflow, timing, event firing
10	infrastructure	`ComponentRegistry`	catalogue, selection, configuration, validation	the nine components	reflection-based property application, dependency validation

#	Role	Component (componentId)	Provides (I)	Requires (D)	Hidden behaviour (B)
11	infrastructure	`AssemblyProfile`	component selection, property values, XML load/save	—	descriptor parsing and writing

The same catalogue is held by `ComponentRegistry` and is visible in the registry source listing, `docs/screenshots/20-code-registry.png`.

3.3 The five characteristics of software components, applied

Characteristic	How this project satisfies it	Evidence
Encapsulation / black-box abstraction	The engine package is package-private; callers see only `QrCodeEncoder`. No public method exposes a Reed–Solomon or masking detail.	`src/main/java/ke/ku/sco310/qr/engine/*` (no `public class` except `QrEngine` / `QrTables`, which are consumed only by the in-house bean)
Reusability	The same nine components are used by two assemblies with two different user interfaces, and the renderers are reused inside the HTTP service per request.	§6.2, `docs/screenshots/03/04/05-gui-*.png`, `15-http-service.png`
Replaceability	Two encoders and three renderers implement identical interfaces; the assembly swaps them at run time (`replaceRenderer`, `replaceEncoder`) without recompiling any caller.	`AssemblyTest` checks `assembly.replace`
Interoperability / standardisation	The contract is the standard JavaBeans model (`java.beans.*`); the third-party ZXing component interoperates through an adapter; artifacts interoperate through PNG, SVG and MIME types.	`docs/screenshots/24-code-zxing-adapter.png`
Composability / independence	Components never reference each other's classes, only interfaces; the facade composes four of them and a listener set. Independent deployment is shown by the JAR + descriptor.	`docs/diagrams/figure-1-component-model.png`

3.4 COP versus OOP in this project

Aspect	Object-oriented view	How this project is *component*-oriented
Basic unit	`QrCode`, `BitBuffer`, `GaloisField` are objects/classes	the unit of *deployment and reuse* is the bean; `PureJavaQrEncoderBean` is dropped into two assemblies untouched
Granularity	fine-grained classes	coarse-grained components behind roles (`encoder`, `renderer`, …)
Reuse	inheritance and library calls	third-party composition: ZXing plugged in behind our interface; artifacts reused by external systems
Interaction	method calls on concrete types	event delegation (`QrCodeGenerationListener`) and bound-property notification (`PropertyChangeEvent`)
Configuration	constructor arguments compiled into the code	design-time properties applied from `config/assembly.xml` through introspection
Evolution	edit class, recompile, redeploy	replace the component named in the descriptor, e.g. `renderer.png.v1` → `renderer.svg.v1`

3.5 Benefits of components realised here

1. **Faster development** — the HTTP front end needed 260 lines of HTTP plumbing and *no* domain logic because encoding, rendering and storage already existed as components.
2. **Reliability through reuse of verified parts** — the composed checks confirm that the same renderer component behaves identically in the GUI and the web service.
3. **Standardisation** — the JavaBeans conventions are what allow the `BeanInspector` tool (225 lines) to describe *any* component without knowing it in advance.
4. **Maintainability** — defects are localised: the interleaving defect described in §9.4 was fixed inside one package-private method without touching any component interface.
5. **Interoperability** — a symbol produced by the in-house encoder is decoded by the ZXing decoder, and vice versa (see the replaceability checks), i.e. third-party composition works in both directions.
6. **Scalability through assembly** — adding a fourth renderer (for example a PDF renderer) means adding one bean and one descriptor line: no existing file changes.

3.6 Component interface incompatibility: the three classic types, and how this design avoids them

Interface incompatibility is the well-known obstacle to component reuse. The three types — and the concrete countermeasures taken in this project — are:

(1) Signature incompatibility (syntactic). The provider's method names, parameter types or exception types do not match what the consumer expects. *Countermeasure:* this suite freezes the contract in the `ke.ku.sco310.qr.api` package (`QrCodeEncoder`, `QrCodeRenderer`, `QrCodeRepository`, `QrCodeGenerationListener`) and never lets an implementation change it; the third-party ZXing engine, whose signatures are completely different (`Encoder.encode(String, ErrorCorrectionLevel, Map<EncodeHintType,?>)` returning a `QRCode`), is adapted by `ZxingQrEncoderAdapterBean`. Both encoders report failures through the same checked exception hierarchy, so callers need no adaptation.

(2) Protocol / behavioural incompatibility (interaction order and state). Even with matching signatures, components may disagree about the *sequence* of calls, blocking behaviour or state expectations. *Countermeasure:* the doc-comments of every interface state pre-conditions and post-conditions (for example "`render` accepts a non-null symbol produced by an encoder of this suite and returns a non-empty artifact whose format matches `getFormatName()`"), and the facade enforces the generation order encode → render → store → notify. The listener contract is one-directional and non-blocking, and the facade isolates a misbehaving listener with a `try`/`catch` so that a faulty listener cannot break the generation protocol.

(3) Semantic / quality-of-service incompatibility (meaning of data). Components may agree on syntax and protocol yet disagree on *what the data means*: units, colour space, coordinate origin or quality guarantees. *Countermeasure:* the suite publishes semantics in metadata and keeps the value object explicit — a `QrCode` always states its version, size, mask, error-correction level and module matrix indexed `[y][x]` with `true` meaning *dark*; a `RenderedArtifact` always declares `formatName`, `fileExtension`, `mimeType` and pixel dimensions so that consumers do not have to guess; the PNG renderer documents that the quiet zone is painted with the background colour and that module centres can be sampled at `(quietZone + x) * moduleSize + moduleSize/2` (this is exactly how the test suite verifies it). Capability metadata (`EncoderInfo`) states the supported modes and version range, so a consumer can test admissibility before calling.

A related, subtler problem is **capability and context mismatch**: the ZXing adapter declares `requiresOptionalLibrary = true` and `ComponentRegistry` asks `OptionalComponentSupport.isZxingAvailable()` *before* instantiating it, so a missing library degrades the deployment (the in-house engine stays wired) instead of producing a `NoClassDefFoundError` at the first call — explicit context dependencies in practice.

---

4. The JavaBeans component model in this project

4.1 The eight steps to create a JavaBean, and where they appear here

Step (course material)	Implementation in this project
1. Package statement as the first line	every file starts with `package ke.ku.sco310.qr.<role>;`
2. Implement `Serializable` for persistence	all nine beans; enforced by a check in `BeanContractTest`
3. Compile with `-d`	`javac -cp lib/core-3.5.3.jar -d out/classes src/main/java/ke/ku/sco310/qr/*/*.java`
4. Create a manifest describing the JAR contents	`manifest/MANIFEST.MF` with a `Java-Bean: True` block per bean class
5. Create the JAR	`java -cp out/classes ke.ku.sco310.qr.tools.ProjectBuilder jar` → `out/dist/qr-component-suite-1.0.0.jar`
6. Check the archive	`jar --list`, `unzip -p ... META-INF/MANIFEST.MF` (`docs/screenshots/14-jar-manifest.png`)
7. Test the bean wrapped in the JAR	run any assembly with `-cp out/dist/qr-component-suite-1.0.0.jar:lib/*`
8. Add the bean into a builder tool	`BeanInspector` plays the role of the builder tool's palette/feature viewer; `PngRendererBeanBeanInfo` supplies the curated metadata a BeanBox would show

4.2 The four property kinds, all implemented

Kind	Definition (course material)	Implementation	Proof
Simple	a private value accessed through `get`/`set`	`PureJavaQrEncoderBean.maximumVersion`, `forcedMaskPattern`; `AsciiRendererBean.border`	`BeanInspector` property table
Bound	listeners are notified whenever the value changes	`PngRendererBean.moduleSize`/`quietZone`/`foregroundColor`, `SvgRendererBean.mergeHorizontalRuns`, `QrHistoryBean.entries`, `QrStudioFacadeBean.defaultErrorCorrection` … all with `PropertyChangeSupport`	`BeanContractTest` "changing moduleSize fires a PropertyChangeEvent"; the GUI preview refreshes on that event
Constrained	a change may be **vetoed**	`QrHistoryBean.maxEntries` (a listener can refuse a reduction); `PureJavaQrEncoderBean.minimumVersion` with `VetoableChangeSupport`	`BeanContractTest` "a listener can veto a capacity reduction"
Indexed	a collection accessed by index	`QrHistoryBean.entries[]` with `getEntries()`, `getEntries(int)`, `setEntries(QrHistoryEntry[])`, `setEntries(int, QrHistoryEntry)` and `fireIndexedPropertyChange`	`BeanInspector` shows `INDEXED yes`; `BeanContractTest` "addEntry fires an IndexedPropertyChangeEvent"

Minimal bound-property implementation, exactly as the course notes prescribe:

```
private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);

public void addPropertyChangeListener(PropertyChangeListener listener) {
    propertyChanges.addPropertyChangeListener(listener);
}

public void setModuleSize(int moduleSize) {
    int old = this.moduleSize;
    this.moduleSize = moduleSize;
    propertyChanges.firePropertyChange("moduleSize", old, moduleSize);   // old + new value
}
```

Constrained property (veto) implementation:

```
public void setMaxEntries(int maxEntries) throws PropertyVetoException {
    int old = this.maxEntries;
    vetoableChanges.fireVetoableChange("maxEntries", old, maxEntries);    // listeners may throw
    this.maxEntries = maxEntries;
    propertyChanges.firePropertyChange("maxEntries", old, maxEntries);    // then the bound event
}
```

4.3 The connection model

The JavaBeans connection model has four roles; all four exist here:

Role	Component
Event source	`QrStudioFacadeBean` — fires `QrCodeGeneratedEvent` after each generation
Event listener	`AuditLogListenerBean` (audit trail), `QrStudioFrame` (status bar), HTTP statistics
Event object	`QrCodeGeneratedEvent extends EventObject` (carries symbol, artifact, stored file, renderer id and duration)
Delegation	`facade.addQrCodeGenerationListener(listener)`; on firing, the source calls the corresponding listener method on every registered listener

Additional connections used:

- **bound-property connections** — UI ← renderer (`moduleSize`, colours), UI ← history (`entries`), HTTP ← audit listener (`processedEventCount`);
- **interface-typed connections** — `facade.setEncoder/setRenderer/setRepository/setHistory`, i.e. the facade only ever stores interfaces.

```
// wiring performed by ComponentRegistry.build(profile)
facade.setEncoder(encoder);
facade.setRenderer(renderer);
facade.setRepository(repository);
facade.setHistory(history);
facade.addQrCodeGenerationListener(listener);   // delegation connection
```

4.4 The deployment model

Packaging follows steps 4–6 of the bean recipe:

- `manifest/MANIFEST.MF` — declares `Main-Class`, `Class-Path`, specification metadata for the JavaBeans component model and a `Java-Bean: True` block for every bean class file;
- `QrStudio.jar` — the standalone desktop application (component classes plus the optional ZXing classes, so it runs without a `lib/` folder);
- `out/dist/qr-component-suite-1.0.0.jar` — the thin deployable unit (118 KB, 51 component classes, 52 entries) plus `lib/` for the optional ZXing component;
- `config/assembly.xml` — the deployment descriptor: component selection per role and the design-time values of their properties. Deployments change by editing this file, never by recompiling, which is exactly the parameter set *P* of the deployment quadruple **D = (C, E, P, L)** used in the course material (C = the component set, E = Java SE 11+, P = the descriptor, L = the machine/workspace).

4.5 Persistence: two mechanisms, as the course requires

1. **Serialization.** `FileSystemRepositoryBean` serialises the list of `QrHistoryEntry` records into `history.ser` (`ObjectOutputStream`) and reads it back with `ObjectInputStream`; the records themselves implement `Serializable` with a no-argument constructor. Verified by "reinstantiating the repository component reads the same history file".
2. **XML bean persistence.** `BeanPersistence` uses `java.beans.XMLEncoder` / `XMLDecoder` to write and rebuild the *whole assembled bean graph* — class names, property values and wiring: `docs/evidence/work/bean-graph.xml` (1.2 KB). This is the mechanism the original Bean Builder used, and it is verified by "the assembled bean graph survives an XML round trip" and "the restored assembly can generate without recompiling". Properties that would bloat the file are excluded declaratively through a BeanInfo (`QrHistoryBeanBeanInfo` marks the indexed history property `transient` for the encoder) — a technique the course notes call "expose only those features you want to expose".

4.6 Introspection, BeanInfo and customisation

Feature descriptor	Where it is used
`BeanDescriptor`	`PngRendererBeanBeanInfo` / `QrHistoryBeanBeanInfo`: display name and short description
`PropertyDescriptor`	display names/descriptions, `expert` flag on `quietZone`, and `setPropertyEditorClass(HexColorPropertyEditor.class)` for both colour properties
`IndexedPropertyDescriptor`	discovered automatically for `QrHistoryBean.entries` (the tool prints `INDEXED yes`)
`EventSetDescriptor`	discovered from `addQrCodeGenerationListener` / `addPropertyChangeListener`; the tool prints the event set and the listener type
`MethodDescriptor`	the tool lists the public business methods (excluding accessors and listener plumbing)
`PropertyEditorSupport`	`HexColorPropertyEditor` converts a `Color` to/from `#RRGGBB`, and offers a tag list for a builder tool's drop-down

Introspection is demonstrated by the project's own tool (`java ke.ku.sco310.qr.tools.BeanInspector <class>`), which prints the bean's required characteristics, its properties with types/read/write/indexed/editor columns, its event sets and its methods — see `docs/screenshots/10-beaninspector-png-renderer.png`, `11-beaninspector-history-facade.png` and `26-bean-feature-matrix.png`.

---

5. The component catalogue, component by component

This chapter documents every component in the suite in the form the course asks for: the role it plays, the interfaces it **provides**, the interfaces it **requires**, the properties it publishes to builder tools, the events it fires, and the behaviour it hides. Each entry ends with the automated checks that hold the component to its contract.

The introspection data below is not hand-written: it is the output of the project's own tool (the `BeanInspector` class directly, see `docs/screenshots/10-beaninspector-png-renderer.png`, `11-beaninspector-history-facade.png` and `26-bean-feature-matrix.png`), which drives `java.beans.Introspector` on the compiled classes.

5.1 Component map

#	Role	Component (id)	Source file	Lines
1	encoder	`PureJavaQrEncoderBean` (`encoder.pure-java.v1`)	`components/PureJavaQrEncoderBean.java`	207
2	encoder	`ZxingQrEncoderAdapterBean` (`encoder.zxing-adapter.v1`)	`components/ZxingQrEncoderAdapterBean.java`	200
3	renderer	`PngRendererBean` (`renderer.png.v1`)	`render/PngRendererBean.java`	207
4	renderer	`SvgRendererBean` (`renderer.svg.v1`)	`render/SvgRendererBean.java`	241
5	renderer	`AsciiRendererBean` (`renderer.ascii.v1`)	`render/AsciiRendererBean.java`	123
6	repository	`FileSystemRepositoryBean` (`repository.file-system.v1`)	`store/FileSystemRepositoryBean.java`	208
7	history	`QrHistoryBean` (`history.bounded-memory.v1`)	`store/QrHistoryBean.java`	233
8	listener	`AuditLogListenerBean` (`listener.audit-log.v1`)	`log/AuditLogListenerBean.java`	175
9	facade	`QrStudioFacadeBean` (`facade.qr-studio.v1`)	`facade/QrStudioFacadeBean.java`	328
10	infrastructure	`ComponentRegistry`	`facade/ComponentRegistry.java`	336
11	infrastructure	`AssemblyProfile` (+ `AssemblyGraphBean`)	`facade/AssemblyProfile.java`	180

`ComponentRegistry.listComponents()` builds this catalogue at run time together with the state of each component, which is how the suite shows that a component may be **declared but unavailable** (see the registry source listing, `docs/screenshots/20-code-registry.png`).

5.2 `PureJavaQrEncoderBean` — the in-house encoder

Role `encoder` · **Identifier** `encoder.pure-java.v1` · **Source** `components/PureJavaQrEncoderBean.java`

Provides (I)	`QrCodeEncoder` (business interface), `BoundPropertySource` (introspection interface), `java.io.Serializable` (deployment interface)
Requires (D)	nothing at run time — only the JDK, which is the point of having an in-house engine
Hidden behaviour (B)	the whole `ke.ku.sco310.qr.engine` package: segment analysis, version selection, Reed–Solomon over GF(2^8), block interleaving, function patterns, masking and the penalty function

Properties (7 discovered, 5 writable)

Property	Type	Kind	Default	Purpose
`componentId`	`String`	simple, read-only	`encoder.pure-java.v1`	stable identity used in evidence records
`encoderInfo`	`EncoderInfo`	simple, read-only	—	capability metadata: provider, version, version range, modes, maximum capacity, third-party flag
`defaultErrorCorrection`	`QrErrorCorrection`	**bound**	`M`	level used when the caller passes `null`
`boostErrorCorrection`	`boolean`	**bound**	`false`	climb to the strongest level that still fits, improving robustness of a printed symbol
`minimumVersion`	`int`	**constrained**	`1`	lowest version selectable; a listener may veto, and the bean vetoes an inconsistent window
`maximumVersion`	`int`	**bound** (clamped)	`40`	highest version selectable; a value below the minimum is clamped
`forcedMaskPattern`	`int`	**bound**	`-1`	pins the mask (0–7) for experiments; `-1` restores automatic penalty-based selection

Events `propertyChange` (from `PropertyChangeSupport`), `vetoableChange` (from `VetoableChangeSupport`).

Code — the complete bean, its four property kinds and the engine delegation is on `docs/screenshots/16-code-encoder-bean.png`; the provided interface it implements is on `docs/screenshots/01-code-provided-interface.png`.

Contract checks `encode()` pre-conditions and post-conditions are asserted by 18 engine checks and by three bean checks:

```
PASS PureJavaQrEncoderBean: public class with a no-argument constructor   public=true
noArgConstructor=true
PASS PureJavaQrEncoderBean: exposes writable properties through set
PASS PureJavaQrEncoderBean: implements Serializable and BoundPropertySource
PASS empty payload is rejected        rejected as expected: QrEncodingException
PASS over-capacity payload is rejected with a clear error
PASS constrained minimumVersion vetoes an inconsistent window
```

5.3 `ZxingQrEncoderAdapterBean` — the replaceable third-party encoder

Role `encoder` · **Identifier** `encoder.zxing-adapter.v1` · **Source** `components/ZxingQrEncoderAdapterBean.java`

Provides (I)	`QrCodeEncoder`, `BoundPropertySource`, `java.io.Serializable` — *exactly the same business interface as the in-house component*
Requires (D)	`com.google.zxing.core` (declared as an **optional** context dependency)
Hidden behaviour (B)	translation of the foreign `com.google.zxing.common.ByteMatrix` result into this suite's `QrCode` value object, charset handling, and the mapping of foreign exceptions onto `QrEncodingException`

This is the component that demonstrates the two ideas the course stresses most: **interface standardisation** (a completely different implementation satisfies the same provided interface) and **third-party composition** (the foreign library is wrapped, never called directly by the rest of the suite). Because the dependency is optional, the component degrades gracefully:

```
encoder  encoder.zxing-adapter.v1   unavailable
         reason: com.google.zxing.core is not on the class path
```

Properties (5 discovered, 3 writable)

Property	Type	Kind	Default	Purpose
`componentId`	`String`	simple, read-only	`encoder.zxing-adapter.v1`	identity
`encoderInfo`	`EncoderInfo`	simple, read-only	—	reports `thirdParty=true`, provider `ZXing ("Zebra Crossing") core 3.5.3`
`characterSet`	`String`	**bound**	`UTF-8`	charset used for byte-mode content
`defaultErrorCorrection`	`QrErrorCorrection`	**bound**	`M`	level used when the caller passes `null`
`boostErrorCorrection`	`boolean`	**bound**	`false`	mirror of the in-house property, so the two components are configured identically

Code `docs/screenshots/24-code-zxing-adapter.png`.

Contract checks the assembly swaps the encoder at run time and proves that the caller cannot tell the difference beyond the metadata:

```
PASS the third-party encoder satisfies the same contract
     expected=https://www.ku.ac.ke/sco310/component-programming
     actual  =https://www.ku.ac.ke/sco310/component-programming
PASS the facade now reports the adapter as its encoder     encoder.zxing-adapter.v1
PASS the optional component declares its context dependency
PASS the assembly detects the optional third-party component   com.google.zxing.core present=true
```

5.4 `PngRendererBean` — raster renderer

Role `renderer` · **Identifier** `renderer.png.v1` · **Source** `render/PngRendererBean.java`

Provides (I)	`QrCodeRenderer` (business), `BoundPropertySource`, `Serializable`
Requires (D)	— (`javax.imageio` is part of the JDK)
Hidden behaviour (B)	pixel geometry `(size + 2 · quietZone) · moduleSize`, run-length rectangle drawing, buffered-image allocation and `ImageIO` encoding

Properties (4 discovered, 4 writable — every property is bound)

Property	Type	Kind	Default	Constraint
`moduleSize`	`int`	bound	`8`	`1..64`
`quietZone`	`int`	bound, **expert**	`4`	`0..16`
`foregroundColor`	`Color`	bound + **custom property editor**	black	`HexColorPropertyEditor`
`backgroundColor`	`Color`	bound + **custom property editor**	white	`HexColorPropertyEditor`

This bean also carries the project's `BeanInfo` demonstration (`PngRendererBeanBeanInfo`, `docs/screenshots/21-code-beaninfo.png`): it publishes display names, marks `quietZone` as an *expert* feature (hidden in a builder tool's basic property sheet) and attaches `HexColorPropertyEditor` to both colour properties, so the tool offers a `#RRGGBB` field with a tag list instead of an opaque `java.awt.Color` value.

Contract checks

```
PASS artifact size follows moduleSize and quietZone     expected=246 actual=246
PASS every module is painted at the right position      0 of 1089 modules mismatched
PASS quiet zone is painted with the background colour
PASS bound moduleSize property resizes the artifact     before=246px after=451px
PASS invalid module size is rejected                    moduleSize must be 1..64
PASS custom property editor parses #RRGGBB              expected=#1B5E20 actual=#1B5E20
```

5.5 `SvgRendererBean` — vector renderer

Role `renderer` · **Identifier** `renderer.svg.v1` · **Source** `render/SvgRendererBean.java`

Provides (I)	`QrCodeRenderer`, `BoundPropertySource`, `Serializable`
Requires (D)	—
Hidden behaviour (B)	SVG document model, absolute millimetre sizing, horizontal run merging, XML escaping

Properties (11 discovered, 6 writable)

Property	Type	Kind	Default	Purpose
`rendererId`, `formatName`, `fileExtension`, `mimeType`	`String`	simple, read-only	`renderer.svg.v1`, `SVG`, `svg`, `image/svg+xml`	interface metadata consumed by the HTTP front end
`moduleSize`	`int`	bound	`8`	raster-equivalent module size
`quietZone`	`int`	bound	`4`	light margin in modules
`foregroundColor`	`String`	bound	`#000000`	vector stroke colour
`backgroundColor`	`String`	bound	`#FFFFFF`	background rectangle
`mergeHorizontalRuns`	`boolean`	bound	`true`	merges adjacent dark modules into one `<rect>`
`physicalWidthMillimetres`	`double`	bound	`0` (meaning "not set")	prints the document with an absolute physical width, as a print shop needs

The run-merging optimisation is measurable and is proven by a matched pair of checks: with the optimisation on the document contains 297 `<rect>` elements, with it off exactly the 577 dark modules are emitted one by one — and both documents rebuild the same module matrix.

```
PASS document declares absolute millimetre dimensions
PASS horizontal run merging reduces the element count     merged=297 single=577
PASS parsing the SVG back reproduces the module matrix    0 module mismatches
```

5.6 `AsciiRendererBean` — console renderer

Role `renderer` · **Identifier** `renderer.ascii.v1` · **Source** `render/AsciiRendererBean.java`

Provides (I)	`QrCodeRenderer`, `BoundPropertySource`, `Serializable`
Requires (D)	—
Hidden behaviour (B)	glyph mapping and the aspect-ratio correction that makes a text symbol recognisable in a terminal

Properties (7 discovered, 3 writable)

Property	Type	Kind	Default	Purpose
`rendererId`, `formatName`, `fileExtension`, `mimeType`	`String`	simple, read-only	`renderer.ascii.v1`, `ASCII`, `txt`, `text/plain`	metadata; the artifact is flagged as textual for the HTTP front end
`darkGlyph`	`String`	bound	██ (two full blocks)	dark module
`lightGlyph`	`String`	bound	two spaces	light module
`border`	`int`	bound	`2`	quiet zone in characters

Contract checks

```
PASS parsing the text art reproduces the module matrix     0 module mismatches
PASS artifact is flagged as textual                        expected=true actual=true
```

5.7 `FileSystemRepositoryBean` — persistence component

Role `repository` · **Identifier** `repository.file-system.v1` · **Source** `store/FileSystemRepositoryBean.java`

Provides (I)	`QrCodeRepository` (required interface of the facade), `BoundPropertySource`, `Serializable`
Requires (D)	the file system, expressed through the configurable `rootDirectory` property — the deployment parameter *P* of **D = (C, E, P, L)**
Hidden behaviour (B)	artifact writing, `ObjectOutputStream` / `ObjectInputStream` history serialisation, file-name sanitising, directory creation

Properties (6 discovered, 2 writable)

Property	Type	Kind	Default	Purpose
`rootDirectory`	`File`	bound	`out/generated`	deployment-specific storage location
`historyFileName`	`String`	bound	`history.ser`	name of the serialised history
`componentId`	`String`	simple, read-only	`repository.file-system.v1`	identity
`historyCount`, `storedArtifactCount`	`int`	simple, read-only	`0`	live counters, bound so that a UI can display them
`historySnapshot`	`List`	simple, read-only	—	defensive copy of the loaded history

Contract checks

```
PASS generation stores an artifact and returns the path
PASS history and audit counters were updated        history=1 audit=1 facade=1
PASS reinstantiating the repository component reads the same history file
PASS clearHistory empties the persisted history     expected=0 actual=0
PASS a new record is persisted immediately          expected=1 actual=1
```

The last check is the important one for component programming: a **newly instantiated component** reads the state written by a previous instance, which is what "deployable independently" means in practice. A second implementation of the same role, `repository.in-memory.v1`, is declared in the catalogue to make the substitution possible.

5.8 `QrHistoryBean` — the indexed, constrained history

Role `history` · **Identifier** `history.bounded-memory.v1` · **Source** `store/QrHistoryBean.java`

Provides (I)	the indexed property `entries` and the constrained property `maxEntries`; `Serializable` (so the facade can hold it)
Requires (D)	—
Hidden behaviour (B)	capacity policy (oldest entry discarded first), overflow accounting, index bounds checking, indexed event firing

Properties (2 discovered, 2 writable) — this is the component that demonstrates the two property kinds the encoder does not have:

Property	Type	Kind	Default	Accessors
`entries`	`QrHistoryEntry[]`	**indexed**	empty	`getEntries()`, `getEntries(int)`, `setEntries(QrHistoryEntry[])`, `setEntries(int, QrHistoryEntry)`, `addEntry(QrHistoryEntry)`
`maxEntries`	`int`	**constrained**	`50`	`getMaxEntries()`, `setMaxEntries(int) throws PropertyVetoException`

`BeanInspector` confirms the indexed nature of the property on the compiled class:

```
PROPERTY       TYPE                READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
entries        QrHistoryEntry[]    yes    yes    yes       normal    -
maxEntries     int                 yes    yes    -         normal    -
```

Events `propertyChange` (+ indexed property change events for `entries`) and `vetoableChange`.

Contract checks

```
PASS addEntry fires an IndexedPropertyChangeEvent       4 events observed
PASS indexed accessor returns the stored element        expected=1 actual=1
PASS capacity policy discards the oldest entry          count=3 discarded=1
PASS a listener can veto a capacity reduction           PropertyVetoException: capacity may not be
reduced
PASS the vetoed value was not applied                   expected=5 actual=5
PASS removing the listener allows the change again      expected=7 actual=7
```

Code `docs/screenshots/18-code-history-bean.png`.

5.9 `AuditLogListenerBean` — the event listener component

Role `listener` · **Identifier** `listener.audit-log.v1` · **Source** `log/AuditLogListenerBean.java`

Provides (I)	`QrCodeGenerationListener` (so the facade can connect it) and read-only statistics properties
Requires (D)	`QrCodeGeneratedEvent` — the event type it consumes, i.e. the *connection* contract with the facade
Hidden behaviour (B)	log-line formatting, console echoing, file appending, ring buffer of recent lines

Properties (6 discovered, 3 writable)

Property	Type	Kind	Default	Purpose
`componentId`	`String`	simple, read-only	`listener.audit-log.v1`	identity
`echoToConsole`	`boolean`	bound	`true`	mirror the audit trail to `System.out`
`verbose`	`boolean`	bound	`false`	include symbol details in each line
`logFile`	`File`	bound	unset	append the audit trail to a file
`processedEventCount`	`int`	simple, read-only, bound	`0`	counter shown in the GUI status bar and in `/stats`
`recentLines`	`List`	simple, read-only	—	last lines, for display

Contract check

```
PASS the facade notifies every connected listener   listeners notified=1 audit events=1
PASS the event object carries the symbol and the artifact
     symbol=QrCode{version=2, size=25, ec=M, mode=Byte, mask=6, ...}
     artifact=RenderedArtifact{format=PNG, 264x264, bytes=5865, by=renderer.png.v1}
```

5.10 `QrStudioFacadeBean` — the composite component

Role `facade` · **Identifier** `facade.qr-studio.v1` · **Source** `facade/QrStudioFacadeBean.java`

The facade is the component that *composes* the others; it is a component in its own right because it publishes its own interface, its own properties and its own events.

Provides (I)	`QrGenerationResult generate(String, QrErrorCorrection)`, `RenderedArtifact renderWith(QrCode, QrCodeRenderer)`, `void addQrCodeGenerationListener(...)`, plus 12 properties
Requires (D)	`QrCodeEncoder`, `QrCodeRenderer`, `QrCodeRepository`, `QrHistoryBean` — four required interfaces, all injected through setters, never instantiated internally
Hidden behaviour (B)	the generation workflow: encode → render → store → record history → fire events → time the operation

Properties (12 discovered, 8 writable) — the four dependency properties are the most important:

Property	Type	Kind	Default	Role
`encoder`	`QrCodeEncoder`	bound	unset	**required interface #1**
`renderer`	`QrCodeRenderer`	bound	unset	**required interface #2**
`repository`	`QrCodeRepository`	bound	unset	**required interface #3**
`history`	`QrHistoryBean`	bound	unset	**required interface #4**
`defaultErrorCorrection`	`QrErrorCorrection`	bound	`M`	level used when the caller passes `null`
`autoSaveArtifacts`	`boolean`	bound	`true`	write each artifact through the repository
`autoSaveHistory`	`boolean`	bound	`true`	append each record to the history
`fileBaseName`	`String`	bound	`qr-code`	stem of generated file names
`componentId`	`String`	simple, read-only	`facade.qr-studio.v1`	identity
`generatedCount`	`int`	simple, read-only, bound	`0`	work counter
`listenerCount`	`int`	simple, read-only	`0`	number of connected listeners

Events `propertyChange` and `qrCodeGeneration` (the delegation event set discovered from `addQrCodeGenerationListener`).

Code `docs/screenshots/19-code-facade.png`.

Contract checks — the failure path is as important as the success path:

```
PASS generating without an encoder fails loudly
     rejected as expected: QrEncodingException: No encoder component is connected:
     call setEncoder(...) before generating
```

5.11 The infrastructure components

`ComponentRegistry` and `AssemblyProfile` are not beans — they are the *assembly tooling* that a component platform needs (the role the original BeanBox or a container plays):

Component	Provides	Requires	Responsibilities
`ComponentRegistry`	`listComponents()`, `listByRole(role)`, `describe(id)`, `build(profile)`, `validate(assembly)`	the nine components, through reflection	catalogue construction, implementation selection per role, **reflection-based property application** (`PropertyCoercion` converts the descriptor's strings to `int`, `boolean`, `double`, `QrErrorCorrection`, `File`, `Color`), dependency validation
`AssemblyProfile`	`getComponentId(role)`, `setComponentId`, `getProperties(role)`, `setProperty`, `load(File)`, `save(File)`	—	the deployment descriptor: which component fills each role and with which property values; XML load/save
`AssemblyGraphBean`	`toXml()`, `restore(assembly)`	`java.beans.XMLEncoder` / `XMLDecoder`	persistence of the **whole assembled bean graph**, the mechanism the original Bean Builder used

`ComponentRegistry.validate(...)` returns a human-readable wiring report, which the test suite asserts on:

```
PASS every declared dependency of the assembly is satisfied
     encoder    -> facade.setEncoder     : OK
     renderer   -> facade.setRenderer    : OK
     repository -> facade.setRepository  : OK
     history    -> facade.setHistory     : OK
     listener   -> facade event source   : OK
     encoder    -> optional ZXing library: PRESENT
```

---

6. Assembling the components: three applications from one component set

6.1 The composition pipeline

`ComponentRegistry.build(profile)` performs the five-stage composition that component programming prescribes. Each stage is separately testable, which is why the verification suite can assert on them independently:

Stage	What happens	Where it fails loudly
1. Catalogue	the registry lists every component it knows, with its role, its provided interface, its required interface and its current availability	a component whose required library is absent is listed as `unavailable` with a reason — it is never silently used
2. Select	one implementation per role is chosen from the profile (`AssemblyProfile`), defaulting to the built-in profile when no file is given	an unknown component id is rejected: `Unknown renderer component id: renderer.does-not-exist.v1`
3. Instantiate	each selected class is created through its **no-argument constructor** — the JavaBeans requirement — by reflection	a non-bean class is rejected
4. Configure	the descriptor's string values are converted to the property types and applied through the setters, with a `PropertyChangeListener` attached	a property a component does not declare is reported, not ignored
5. Validate	every required interface of every component is checked against the wired graph	`No encoder component is connected: call setEncoder(...) before generating`

The configuration stage is the one that makes the project *component oriented* rather than merely object oriented: property values live in `config/assembly.xml`, and changing them re-deploys the same compiled classes.

```
<qr-assembly name="default">
  <component role="encoder"    id="encoder.pure-java.v1"/>
  <component role="renderer"   id="renderer.png.v1"/>
  <component role="repository" id="repository.file-system.v1"/>
  <component role="history"    id="history.bounded-memory.v1"/>
  <component role="listener"   id="listener.audit-log.v1"/>
  <component role="facade"     id="facade.qr-studio.v1"/>
  <property role="renderer" name="moduleSize"  value="10"/>
  <property role="renderer" name="quietZone"   value="4"/>
  <property role="renderer" name="foregroundColor" value="#101820"/>
  <property role="history"  name="maxEntries"  value="50"/>
  <property role="facade"   name="fileBaseName" value="qr-code"/>
</qr-assembly>
```

The same file generates the deployment report printed at start-up by both front ends:

```
facade[encoder=encoder.pure-java.v1, renderer=renderer.png.v1,
       repository=repository.file-system.v1, history=history.bounded-memory.v1, listeners=1]
renderer=PNG: moduleSize=10px, quietZone=4 modules, fg=#101820, bg=#FFFFFF
```

6.2 Two front ends, one component set

Assembly	Class	What it adds	Evidence
Desktop (Swing)	`ui/QrStudioGuiAssembly` + `ui/QrStudioFrame`	a compact form for the payload and output format, a preview that follows renderer replacement, an indexed history view	`docs/screenshots/03/04/05-gui-*-assembly.png`
HTTP service	`web/QrHttpServiceComponent`	one URL per symbol with the renderer chosen per request, plus statistics, history and health endpoints	`docs/screenshots/15-http-service.png`

The two front ends contain **no encoding, rendering or storage logic at all**. They only configure and call components:

- the GUI calls `assembly.generate(...)` and subscribes to `BoundPropertySource` of the renderer (`frame.propertyChange(...)` refreshes the preview when the renderer reports a property change);
- the HTTP service resolves the `format` query parameter to a renderer component per request:

```
QrCodeRenderer renderer = "svg".equalsIgnoreCase(format)  ? new SvgRendererBean()
                        : "ascii".equalsIgnoreCase(format) ? new AsciiRendererBean()
                        : new PngRendererBean();
assembly.replaceRenderer(renderer);          // no caller recompilation
QrGenerationResult result = assembly.generate(text, level);
```

6.3 Replaceability, demonstrated rather than claimed

Replaceability is the property that separates a component suite from a library, so the project tests it four times over:

Replacement	What is swapped	Proof
Renderer → SVG	`renderer.svg.v1` for `renderer.png.v1`	`PASS the assembly runs with a different renderer component → expected=SVG actual=SVG`
Renderer → ASCII	`renderer.ascii.v1` for the SVG component	`PASS a third renderer drops in without recompiling → expected=ASCII actual=ASCII`
Encoder → ZXing	the entire in-house engine for the third-party engine	`PASS the third-party encoder satisfies the same contract`
Both at once	encoder and renderer	`PASS all three renderers produced the same symbol → version 5, mask 7 for PNG, SVG and ASCII`

The last check is the strongest single piece of evidence in the project: **three different renderer components and two different encoder components produce mutually consistent symbols**, because they communicate only through the shared value objects `QrCode` and `RenderedArtifact`.

The GUI shows the same replacement interactively: choosing "SVG" in the *Output format* combo box calls `assembly.replaceRenderer(...)` on the running application, and the status bar reports the new component id and its configuration — the running application changes component without restarting (`docs/screenshots/04-gui-svg-assembly.png`).

6.4 Persistence of the assembly itself

Two mechanisms are implemented, both from the course material:

1. **Descriptor persistence** — `config/assembly.xml` is loaded and saved by `AssemblyProfile` (XML, human editable), verified by `PASS the assembly profile survives an XML round trip`.
2. **Bean-graph persistence** — `AssemblyGraphBean` uses `java.beans.XMLEncoder` / `XMLDecoder` to write the *assembled object graph* (class names, property values, wiring) and rebuild it: `docs/evidence/work/bean-graph.xml`. The suite proves that the restored graph can generate a symbol without recompilation:

```
PASS the assembled bean graph survives an XML round trip
PASS design-time property values survive the round trip     moduleSize=8 xmlBytes=1244
PASS the restored assembly can generate without recompiling  artifact written by the restored graph
```

The XML graph also demonstrates the *"expose only what you want to expose"* technique: the `QrHistoryBeanBeanInfo` marks the indexed `entries` property as transient for the encoder, so a large history is not duplicated inside the graph while remaining visible to builder tools.

6.5 Screen-by-screen: what the running desktop application shows

`docs/screenshots/03-gui-png-assembly.png` (and the SVG/ASCII variants) is a screenshot of the real program, not a mock-up: the frame is built by `QrStudioGuiAssembly --screenshot`, generating with the demonstration payload and painting itself into an image.

Region of the window	Component behind it
Payload	input to `facade.generate(...)`; the error-correction level is the one configured in `assembly.xml`
Output format combo box	`AssemblyProfile` role `renderer`; changing it calls `assembly.replaceRenderer(...)`
Generate / *Save as…*	`facade.generate(...)` / `facade.renderWith(...)`
Rendered artifact panel	the `RenderedArtifact` returned by the renderer component; PNG and SVG are previewed as images (the SVG preview is rasterised through the PNG renderer, while *Save as…* writes the SVG source), and the image is scaled to fit with nearest-neighbour interpolation so the module edges stay crisp
History panel	`historyArea.setText(history.describeEntries())`, refreshed by the indexed property event
Status bar (left)	the symbol's own metadata: version, level, mask, mode, module count
Status bar (centre)	generation duration and the file written by the repository component

Because the preview panel listens to the renderer's bound properties and the history panel listens to the indexed property, the window is a *live view of the component graph*: replacing the renderer component redraws the artifact, and every generation updates the history.

---

7. The encoding logic, in detail

This chapter explains the logic that the encoder components hide behind `QrCodeEncoder`. Everything described here lives in the package-private `ke.ku.sco310.qr.engine` package, and every number quoted was produced by running the compiled code (the worked examples come from instrumentation of the real engine (`ke.ku.sco310.qr.test.MaskComparisonTool`), and the tables are cross-checked against ZXing in `docs/evidence/table-cross-check.csv`).

7.1 The five pipeline stages

`QrEngine.encode(...)` is organised exactly like clause 8 of ISO/IEC 18004:

Stage	Clause	Method	What it decides
1. Data analysis	8.3–8.4	`QrSegment.makeSegments`	which encoding mode is most compact for the payload
2. Data encoding	8.4	`QrSegment.makeNumeric/Alphanumeric/Bytes`	the bit stream of the payload
3. Error correction	8.5	`ReedSolomon.computeRemainder`	one Reed–Solomon block per interleaving block
4. Structure finalisation	8.6	`QrEngine.addEccAndInterleave`	block split and codeword interleaving
5. Module placement	8.7–8.9	`drawFunctionPatterns`, `drawCodewords`, `applyMask`, `drawFormatBits`, `drawVersionBits`	the final matrix, including the mask

`docs/diagrams/figure-4-encoding-pipeline.png` draws this pipeline with the class that implements each stage.

7.2 Stage 1–2: mode analysis and bit-level encoding

The payload chooses its mode by inspection — the most compact representation wins:

Mode	Indicator	Character set	Payload bits	Character-count bits (v1–9 / v10–26 / v27–40)
Numeric	`0001`	`0-9`	3 digits → 10 bits, 2 → 7 bits, 1 → 4 bits	10 / 12 / 14
Alphanumeric	`0010`	`0-9 A-Z $ % * + - . / :` `space` (45 symbols)	2 chars → 11 bits, 1 → 6 bits	9 / 11 / 13
Byte	`0100`	arbitrary data (UTF-8 in this suite)	8 bits per byte	8 / **16** / **16**

The character-count widths are declared as data on the enum, so a table error would be visible in one place:

```
NUMERIC(0x1, new int[]{10, 12, 14}),
ALPHANUMERIC(0x2, new int[]{9, 11, 13}),
BYTE(0x4, new int[]{8, 16, 16});
```

Two details are easy to get wrong and are called out here because the project's own test suite caught one of them (see §9.4):

- the byte-mode counter is **8 bits for versions 1–9 but 16 bits for versions 10–40**, which is why the reported maximum capacity is 2 953 bytes at level L (version 40) and not 2 956;
- numeric groups are padded to the left with zeros — `Integer.parseInt("012")` is `12`, encoded in the 10 bits `0000001100`.

Mode selection is proven by three checks and by a capacity comparison:

```
PASS numeric payload selects numeric mode       expected=Numeric actual=Numeric
PASS upper-case payload selects alphanumeric    expected=Alphanumeric actual=Alphanumeric
PASS mixed case payload selects byte mode       expected=Byte actual=Byte
PASS compact mode lowers the symbol version     numeric v1 vs alphanumeric v1
```

7.3 Stage 3: version selection

The version is chosen by the smallest symbol whose data capacity, **in bits**, can hold the whole bit stream (mode indicator + character count + payload, plus the terminator). Capacity comes from `QrTables.numDataCodewords(version, level) · 8`,

so it always matches the table used for the error-correction blocks:

```
while (true) {
    usedBits = totalBits(segments, version);
    int capacityBits = QrTables.numDataCodewords(version, level) * 8;
    if (usedBits <= capacityBits) break;
    if (version >= maximumVersion) throw new QrEncodingException(producerId, ...);
    version++;
}
```

If `boostErrorCorrection` is set, the level is then raised as far as the *same* symbol still allows — a robustness increase that costs no extra modules. If the payload does not fit even at version 40, the component fails with a message that names the requirement and the capacity:

```
Payload of 200 characters does not fit version 1 at level H
(needs 1113 bits, capacity 72 bits). Shorten the payload or lower the error-correction level.
```

The capacity model is validated exhaustively: for all **160** version/level combinations the encoder is given the *largest* byte payload that fits and the produced symbol is decoded again — `160 decoded, 0 failed` (`docs/evidence/capacity-matrix.csv`).

7.4 Stage 4a: the bit stream — worked example

The example reproduced below is the one used in ISO/IEC 18004 Annex I, so the expected codewords can be checked against the published ones rather than against ourselves.

Payload `01234567`, **version 1**, **level M** (16 data codewords, 10 ECC codewords, 26 total).

Field	Bits	Value
Mode indicator (numeric)	4	`0001`
Character count (8 characters)	10	`0000001000`
`012` → 12	10	`0000001100`
`345` → 345	10	`0101011001`
`67` → 67	7	`1000011`
data bits so far	**41**	
Terminator (up to 4 zero bits)	4	`0000`
Padding to the next codeword boundary	3	`000`
Pad codewords `0xEC`, `0x11`, alternating to 128 bits	80	`11101100 00010001 …`

The resulting data codewords, in hexadecimal, are

```
10 20 0C 56 61 80 EC 11 EC 11 EC 11 EC 11
```

which is exactly the sequence printed in the standard's worked example (`00010000 00100000 00001100 01010110 01100001 10000000 11101100 00010001 …`). The Reed–Solomon stage then produces

```
A5 24 D4 C1 ED 36 C7 87 2C 55
```

and this too matches the published error-correction codewords of that example (`10100101 00100100 11010100 11000001 11101101 00110110 11000111 10000111 00101100 01010101`). Because the interleaved codeword stream is byte-identical

to the reference, the bitmap placement, format information and masking stages are the only remaining variables — and those are verified separately by decoding the produced symbols with an independent decoder.

Two more worked examples from the same instrumentation, showing the other two modes:

Payload	Level	Mode	Data codewords	Error-correction codewords	Chosen mask
`HELLO WORLD` (v1, 11 chars)	Q	Alphanumeric	`20 5B 0B 78 D1 72 DC 4D 43 40 EC 11 EC`	`A8 48 16 52 D9 36 9C 00 2E 0F B4 7A 10`	0
`https://www.ku.ac.ke` (v2, 20 bytes)	Q	Byte	`41 46 87 47 47 07 33 A2 F2 F7 77 77 72 E6 B7 52 E6 16 32 E6 B6 50`	`19 B7 0C 58 76 70 DD 06 DA 08 0E 36 F5 DD 72 51 CF 12 33 18 8C 1C`	4

7.5 Stage 3: Reed–Solomon over GF(2⁸)

Error correction is the part of QR encoding that most implementations get subtly wrong, so the implementation is deliberately explicit. `GaloisField` builds the log/antilog tables for the field generated by the primitive polynomial $\mathbf{x^8 + x^4 + x^3 + x^2 + 1}$ **(`0x11D`)** once, in a static initialiser:

```
int x = 1;
for (int i = 0; i < 255; i++) {
    EXP[i] = x;  LOG[x] = i;
    x <<= 1;
    if ((x & 0x100) != 0) x ^= 0x11D;    // reduce modulo the primitive polynomial
}
```

Multiplication is then a logarithm addition, `EXP[LOG[a] + LOG[b]]`, with the `512`-entry `EXP` table making the doubling of the table unnecessary.

`ReedSolomon.computeDivisor(degree)` builds the generator polynomial $(x - \alpha^0)(x - \alpha^1)\ldots(x - \alpha^{(degree-1)})$; `computeRemainder(data, degree)` performs the polynomial division with a `degree`-byte shift register:

```
for (byte b : data) {
    int factor = (b & 0xFF) ^ remainder[0];
    System.arraycopy(remainder, 1, remainder, 0, degree - 1);
    remainder[degree - 1] = 0;
    for (int i = 0; i < degree; i++) remainder[i] ^= GaloisField.multiply(divisor[i], factor);
}
```

The remainder *is* the sequence of error-correction codewords. The register already performs the `x^degree` shift, so feeding only the data codewords is correct — feeding zero-padded placeholders as well is the defect documented in §9.4.

Code `docs/screenshots/02-code-reed-solomon.png`.

7.6 Stage 4b: block splitting and interleaving

For versions and levels where a symbol has more than one block, the data codewords are split into `numErrorCorrectionBlocks` blocks and the ECC codewords of each block are interleaved with the data codewords. The implementation follows clause 8.6 literally, and the two phases are deliberately kept apart:

```
phase 1: for column = 0 … maxDataLen-1:  for block = 0 … n-1: emit block[column]
phase 2: for column = 0 … blockEccLen-1: for block = 0 … n-1: emit blockEcc[column]
```

Blocks are "short" (`shortBlockLen`) or "long" (`shortBlockLen + 1`), and the short blocks contribute **no codeword at all** in the last data column. Interleaving naively — walking data and ECC columns together — produces a matrix that looks plausible but decodes to garbage; the automated suite caught precisely that during development (`§9.4` , defect 1).

The block arithmetic is stated once and used everywhere:

```
int numBlocks      = QrTables.numErrorCorrectionBlocks(level, version);
int blockEccLen    = QrTables.eccCodewordsPerBlock(level, version);
int rawCodewords   = QrTables.totalCodewords(version);
int shortBlockLen  = rawCodewords / numBlocks;
int numShortBlocks = numBlocks - rawCodewords % numBlocks;
```

7.7 Stage 5a: function patterns

Before any data module is placed, the reserved patterns are drawn and flagged in a parallel `isFunction` matrix so that later stages can never overwrite them:

Pattern	Position	Source
Finder patterns + separators	`(3,3)` , `(size−4,3)` , `(3,size−4)`	drawn by `drawFinderPattern` , 7×7 with a light separator
Timing patterns	row 6 and column 6	alternating dark/light, `drawFunctionPatterns`
Alignment patterns	`QrTables.alignmentPatternPositions(version)`	5×5 patterns excluding the three finder corners
Dark module	`(8, size−8)`	always dark, written with the format information
Format information	around the two finder patterns	15 bits, BCH(15,5)-protected
Version information	two 6×3 blocks, version ≥ 7	18 bits, BCH(18,6)-protected

7.8 Stage 5b: codeword placement

The final codewords are placed in the two-module-wide **zig-zag** order of clause 8.7: columns are visited in pairs from right to left, skipping the vertical timing column (6), and alternating upward and downward. Only non-function modules receive bits:

```
for (int right = size - 1; right >= 1; right -= 2) {
    if (right == 6) right = 5;
    for (int vert = 0; vert < size; vert++) {
        for (int j = 0; j < 2; j++) {
            int x = right - j;
            boolean upward = ((right + 1) & 2) == 0;
            int y = upward ? size - 1 - vert : vert;
            if (!isFunction[y][x] && i < data.length * 8) {
                modules[y][x] = getBit(data[i >>> 3], 7 - (i & 7));
                i++;
            }
        }
    }
}
```

7.9 Stage 5c: mask selection and the penalty function

All eight mask patterns of the standard are implemented as the specification's formulas:

Mask	Condition to invert `(x, y)`	Row/column it favours
0	`(x + y) % 2 == 0`	checkerboard
1	`y % 2 == 0`	horizontal stripes
2	`x % 3 == 0`	vertical stripes
3	`(x + y) % 3 == 0`	diagonal
4	`(x/3 + y/2) % 2 == 0`	large blocks
5	`x·y % 2 + x·y % 3 == 0`	dense
6	`(x·y % 2 + x·y % 3) % 2 == 0`	dense, mixed
7	`((x + y) % 2 + x·y % 3) % 2 == 0`	dense, mixed

The best mask is the one with the **lowest penalty**, scored by the four rules of clause 8.8.2 with weights `N1 = 3`, `N2 = 3`, `N3 = 40`, `N4 = 10`:

Rule	Penalty
1 — a run of 5+ modules of the same colour	`3 + (runLength − 5)`
2 — every 2×2 block of the same colour	`3`
3 — a `1:1:3:1:1` finder-like pattern with four light modules on one side	`40`
4 — deviation of the dark-module ratio from 50 %	`10 · k`, where `k` is the number of 5 % steps outside the target

The engine evaluates each candidate, writes the corresponding format bits, scores the matrix, then undid the candidate (XOR is its own inverse) before trying the next:

```
for (int candidate = 0; candidate < 8; candidate++) {
    engine.applyMask(candidate);
    engine.drawFormatBits(candidate, level);
    int penalty = engine.getPenaltyScore();
    if (penalty < bestPenalty) { bestPenalty = penalty; mask = candidate; }
    engine.applyMask(candidate);           // undo
}
```

Penalty scores are observable in the report instead of plausible:

```
payload 01234567  v1-M   mask 0   penalty 1057      (the chosen mask)
                         mask 1   penalty 1253
                         mask 2   penalty 1117   …   mask 7 penalty 1126
```

Because rule 3 leaves room for interpretation at the symbol border (the standard's Figure 24 amendment treats the surrounding quiet zone as light modules), a different implementation can pick a different mask for the same payload. §9.6 quantifies that divergence honestly rather than pretending the implementations are bit-identical.

7.10 Format and version information

The 15-bit format field carries the 2-bit error-correction indicator (the `formatBits` value of `QrErrorCorrection`: L = `01`, M = `00`, Q = `11`, H = `10`) and the 3-bit mask number, protected by a BCH(15,5) code and finally XOR-ed with the constant `0x5412`:

```
int data = level.getFormatBits() << 3 | mask;
int rem = data;
for (int i = 0; i < 10; i++) rem = (rem << 1) ^ ((rem >>> 9) * 0x537);
int bits = (data << 10 | rem) ^ 0x5412;
```

Versions ≥ 7 additionally carry an 18-bit version field, generated with the generator polynomial `0x1F25` and placed in two 6×3 blocks. Both fields are written into the `isFunction` matrix, so data placement can never collide with them.

Note the deliberate design of the level enum: the **declaration order** (L, M, Q, H) matches the row order of the error-correction tables, so `ECC_CODEWORDS_PER_BLOCK[ecl.ordinal()][version]` is the whole indexing scheme — one fewer chance of an off-by-one error.

7.11 Rendering logic

Rendering is a pure function from the `QrCode` matrix to an artifact, which is why all three renderers can be verified by *reconstructing the matrix from the artifact*:

Renderer	Geometry	Reconstruction proof
PNG	`pixelSize = (size + 2·quietZone) · moduleSize`; dark modules are drawn as horizontal runs with `fillRect`	every pixel of all 1 089 modules of a v5 symbol is sampled and compared: `0 of 1089 modules mismatched`
SVG	one `<rect>` per module, or one per *run* when `mergeHorizontalRuns` is on; `width` / `height` in millimetres when `physicalWidthMillimetres` is set	the document is parsed back into a matrix: `0 module mismatches`, and the element count halves (`297` vs `577`)
ASCII	two glyphs per module with a configurable border	the text is parsed back into a matrix: `0 module mismatches`

The ASCII renderer's glyph default (`██` for dark, two spaces for light) is the aspect-ratio correction that makes a text symbol square in a terminal whose character cell is about twice as tall as it is wide.

7.12 Complexity

Operation	Cost	Note
Bit stream assembly	O(payload)	one pass, growable `BitBuffer`
Reed–Solomon	O(dataCodewords · degree)	26–3 700 codewords in practice
Matrix construction	O(size²)	`size = 4·version + 17`, at most 177×177
Mask selection	8 · O(size²)	8 · 31 329 module visits for version 40
PNG rendering	O(pixels)	≈ 1.6 M pixels for version 40 at 10 px per module
SVG rendering	O(dark modules)	halved by run merging

The whole suite is single-threaded and allocation-light; a version-5 symbol is generated, rendered, persisted and logged in 60–140 ms on the test machine, as printed in the status bar of every GUI screenshot.

---

8. Guide to the screenshots, code listings and figures

8.1 Source-code screenshots

Every listing is generated from the real files in `src/` with syntax highlighting (Pygments, `monokai`): no listing was retyped, so the screenshots cannot drift from the code.

Screenshot	File shown	Why it is in the report
`01-code-provided-interface.png`	`api/QrCodeEncoder.java`	the provided interface all encoders implement, with pre/post-conditions in the Javadoc
`02-code-reed-solomon.png`	`engine/ReedSolomon.java`	the shift-register polynomial division that generates the ECC codewords
`16-code-encoder-bean.png`	`components/PureJavaQrEncoderBean.java`	a complete bean: four property kinds, delegation, engine call
`17-code-png-renderer.png`	`render/PngRendererBean.java`	bound properties, validation, run-length drawing
`18-code-history-bean.png`	`store/QrHistoryBean.java`	the **indexed** property and the **constrained** `maxEntries` with veto
`19-code-facade.png`	`facade/QrStudioFacadeBean.java`	composition, event firing, timing
`20-code-registry.png`	`facade/ComponentRegistry.java`	catalogue → select → instantiate → configure → validate
`21-code-beaninfo.png`	`render/PngRendererBeanBeanInfo.java`	feature descriptors, expert flag, custom property editor
`22-code-swing-assembly.png`	`ui/QrStudioFrame.java`	the desktop front end: bound-property listeners, live preview
`24-code-zxing-adapter.png`	`components/ZxingQrEncoderAdapterBean.java`	third-party composition behind our own interface
`25-code-verification.png`	`test/EngineReferenceTest.java`	how the independent-oracle checks are written

8.2 Running-program screenshots

Screenshot	Command that produced it	What to look for
`03-gui-png-assembly.png`	`java … QrStudioGuiAssembly --format PNG --screenshot …`	payload and format controls, live PNG preview, indexed history, symbol identity in the status bar
`04-gui-svg-assembly.png`	same, `--format SVG`	the renderer component replaced on the running application
`05-gui-ascii-assembly.png`	same, `--format ASCII`	a textual artifact rendered into the same preview panel (card switch)
`10-beaninspector-png-renderer.png`	`… BeanInspector … PngRendererBean`	introspection: bean characteristics, properties, editors, event sets, methods
`11-beaninspector-history-facade.png`	`… BeanInspector … QrHistoryBean QrStudioFacadeBean`	`INDEXED yes` on `entries`, the `vetoableChange` event set, the `qrCodeGeneration` delegation event set
`12-verification-transcript.png`	`java -cp out/classes ke.ku.sco310.qr.tools.ProjectBuilder test`	the verification transcript, ending in `108 checks, 108 passed, 0 failed`
`13-build-and-package.png`	`java -cp out/classes ke.ku.sco310.qr.tools.ProjectBuilder all`	compile → verify → package, driven entirely by Java
`14-jar-manifest.png`	`unzip -p …jar META-INF/MANIFEST.MF`	the deployment interface: `Java-Bean: True` for all nine beans
`15-http-service.png`	`java -cp out/classes:lib/* ke.ku.sco310.qr.web.QrHttpServiceComponent --port 8210` + `curl`	`/qr` response headers (`X-qr-version`, `X-qr-mask`, `X-qr-renderer`), `/stats`, `/history`
`26-bean-feature-matrix.png`	`… BeanInspector …` (four beans)	the feature matrix across components in one view
`http-service-symbol.png`	`curl -o … '/qr?text=…&format=png'`	the artifact the web service returned

8.3 Figures

Figure	Content
`figure-1-component-model.png`	the nine components, their roles, provided and required interfaces, and the facade as the composition point
`figure-2-connection-model.png`	the JavaBeans connection model in this project: event source, listener, event object, delegation, plus bound-property and interface-typed connections
`figure-3-deployment-model.png`	**D = (C, E, P, L)**: the component set, the execution environment, the descriptor parameters and the deployment location
`figure-4-encoding-pipeline.png`	the five ISO/IEC 18004 stages with the class that implements each one
`figure-5-verification-strategy.png`	how each group of checks is anchored to an independent oracle or to a reconstruction
`figure-6-artifact-gallery.png`	a contact sheet of generated artifacts (PNG, SVG, ASCII) with their recovered payloads

8.4 Generated artifacts submitted as evidence

Evidence	Content
`docs/evidence/gallery/`	the 20 raster artifacts shown in the contact sheet (including the 10-step version ladder and the four error-correction levels) plus 4 SVG and 4 ASCII artifacts
`docs/evidence/gallery/history.ser`	a history file written by the repository component, readable by a fresh instance
`docs/evidence/work/bean-graph.xml`	the assembled bean graph persisted with `XMLEncoder`
`docs/evidence/work/assembly-roundtrip.xml`	an assembly profile written and re-read by `AssemblyProfile`

---

9. Verification: how the claims in this report were tested

A component suite is only as good as the evidence that it works. This chapter describes the verification strategy, the results, the defects the strategy caught, and — importantly — the two places where this implementation legitimately differs from another conforming implementation.

9.1 Strategy

Four independent techniques are used, because each one catches a different class of error:

Technique	What it catches	Where it is used
Independent oracle	an error in our own tables or encoding logic	every produced symbol is decoded by **ZXing**, a completely separate implementation; the 160 ISO/IEC 18004 parameter combinations are compared cell by cell with ZXing's tables
Round-trip / reconstruction	errors in placement, masking, rendering or persistence	payload → symbol → PNG/SVG/ASCII → parse back → matrix → decode → payload must return unchanged
Exhaustive sweeps	boundary errors that a single example misses	all 40 versions × 4 levels: 160 capacity cases, 160 table comparisons, 8 forced masks, 60 payload×level decodes
Negative controls	a test that passes for the wrong reason	inputs that *must* fail are asserted to fail, with the exact exception and message; one artifact is deliberately corrupted to prove the decoder really rejects it

The suite is written as an in-house harness (`test/TestHarness.java`, 200 lines) rather than JUnit, so the project has **no test-framework dependency** and the whole verification runs from `ProjectBuilder test`. The harness records, for every check, the group, the name, the outcome, the observed value and the elapsed time; the runner writes those records straight into the evidence folder:

```
[evidence] wrote docs/evidence/test-report.txt (13448 chars)
[evidence] wrote docs/evidence/test-results.csv (13918 chars)
[evidence] wrote docs/evidence/test-results.md (15064 chars)
[evidence] wrote docs/evidence/table-cross-check.csv (6450 chars)
[evidence] wrote docs/evidence/capacity-matrix.csv (5109 chars)
[evidence] wrote docs/evidence/renderer-matrix.csv (231 chars)
[evidence] wrote docs/evidence/artifact-gallery.csv (4155 chars)
checks: 108, failures: 0
```

9.2 Where the truth comes from

Nothing in the evidence folder is asserted from memory. Each file has a defined provenance:

Evidence file	Produced by	Independent anchor
`test-report.txt`	`VerificationRunner`	ZXing decoder, ZXing tables, ZXing encoder
`test-results.csv` / `.md`	same run, machine-readable / human-readable	—
`table-cross-check.csv`	engine vs ZXing parameter tables	ZXing `Version` tables
`capacity-matrix.csv`	the largest payload each version/level accepts, decoded back	ZXing decoder
`renderer-matrix.csv`	PNG/SVG/ASCII artifacts rebuilt into matrices	matrix equality with the encoder output
`artifact-gallery.csv`	20 files with SHA-256 and the payload decoded from each	ZXing decoder + SHA-256
`mask-selection-comparison.csv`	mask choice vs ZXing over 104 payload/level pairs	ZXing encoder
`work/bean-graph.xml`, `work/assembly-roundtrip.xml`	XML persistence of graph and profile	re-read by a fresh instance

The oracle is used only as a *verifier*: `ZxingVerify` is the only class in the project that touches the third-party library for testing, and it is compiled into `out/test-classes`, never into the shipped JAR.

9.3 Results

```
TOTAL 108 checks, 108 passed, 0 failed
ISO/IEC 18004 parameter table cross-check: 160 version/level combinations, 0 mismatches
```

Group	Checks	What it establishes
`engine`	18	tables match ZXing; 60/60 payloads decode; mode selection; determinism; 8/8 masks decode; metadata; capacity; error paths
`renderer.png`	6	geometry, MIME type, decodability, quiet zone colour, per-module pixel accuracy, bound resize
`renderer.svg`	5	millimetre sizing, namespace, run merging, matrix reconstruction, optimisation round trip
`renderer.ascii`	2	matrix reconstruction, textual flag
`renderer` (cross-cutting)	6	three implementations of one role, identity in artifacts, property editor, invalid inputs, corrupted-artifact control
`beans` (+ sub-groups)	48	every bean is a valid bean; bound, constrained, indexed and introspection behaviour; events; persistence; optional dependency
`assembly.*`	30	catalogue, wiring, end-to-end generation, replacement, negative controls, repository round trip, history policy
`gallery`	3	the third-party adapter, artifact writing, three formats present

The headline numbers, each traceable to a file:

Claim	Value	Evidence
Parameter tables identical to ZXing	160 / 160	`table-cross-check.csv`
Payloads recovered exactly (incl. accents, CJK, emoji)	60 / 60	`test-report.txt`
Maximum-capacity symbols decoded	160 / 160	`capacity-matrix.csv`
Forced masks that decode	8 / 8	`test-report.txt`
Artifacts with digests	28	`artifact-gallery.csv`
Checks	108 / 108	`test-results.csv`

9.4 Defect log — what the verification caught

These are the real defects found while building and verifying the suite. They are documented because they show the difference the evidence made: three of them would have shipped as silently wrong QR codes.

Defect 1 — Reed–Solomon input and interleaving order (critical). The first implementation fed the shift register `data + zero-padded ECC placeholders` instead of just the data codewords, and interleaved the codewords column-by-column across the *combined* data and ECC sequence. The generated matrices looked like QR codes and passed a naive "is it 21×21 with three finder patterns" check, but no decoder could read them. *Cause* the register in a polynomial-division implementation already performs the `x^degree` shift, so the placeholders are a double shift. *Fix* `ReedSolomon.computeRemainder(dataPart, blockEccLen)` with the data part only, and a two-phase interleave (all data columns, then all ECC columns), with short blocks contributing nothing in the last data column. *Regression test* `PASS payloads survive the ZXing decode round trip (60 decoded, 0 failed)` and the 160-case capacity sweep — neither can pass with this bug present.

Defect 2 — byte-mode character-count width for versions 10–40 (high). The character-count field was hard-coded to 8 bits for byte mode. The standard specifies 8 bits for versions 1–9 but **16 bits** for versions 10–40. *Symptom* symbols from version 10 upward decoded to garbage or failed to decode at all; capacity for those versions was reported 8 bits too high. *Fix* the width is now data on `QrEncodingMode` (`new int[]{8, 16, 16}`) and `byteCapacity(...)` derives the maximum payload from it, so the reported maximum at level L is exactly the specification's 2 953 bytes. *Regression test* `PASS maximum byte capacity reported for level L — expected=2953 actual=2953` plus the 160-case capacity matrix.

Defect 3 — numeric mode bit widths (medium). Three-digit numeric groups were emitted in 11 bits instead of 10, and singleton groups in 5 bits instead of 4. *Symptom* numeric payloads were one bit per group too long, which could push a payload into a larger version than necessary, and made the stream unreadable to a conforming decoder once the error was not compensated. *Fix* `take == 3 ? 10 : (take == 2 ? 7 : 4)`. *Regression test* `PASS numeric payload selects numeric mode`, `PASS compact mode lowers the symbol version`, and the numeric entries of the 60-case decode sweep.

Defect 4 — the screenshot path blocked the Swing event dispatch thread (medium). `QrStudioGuiAssembly --screenshot` did its work inside `SwingUtilities.invokeAndWait(...)` and then called `Thread.sleep(400)` inside that block to let the layout settle. Sleeping on the event dispatch thread forbids the queued re-layout and repaint events from running, so the captured window was painted with the *pre-generation* layout — the symbol identity label was still sized for the placeholder text `-` and was therefore rendered as `...`. *Fix* three phases: build and generate on the EDT, wait **off** the EDT, then paint on the EDT again. *Effect* the symbol metadata now appears correctly in the status bar of `docs/screenshots/03-gui-png-assembly.png` (`v7-M mask 2 Byte 45x45`).

Defect 5 — settings panel collapsed and preview cropped (medium, presentation). Two layout defects in the desktop assembly: `GridBagLayout` centres its grid when the container is taller than its content, so the settings panel floated in the middle of the window; and the artifact preview was placed in a scroll pane that clipped large symbols (a version-7 symbol at 10 px per module is 450 px wide). *Fix* the settings panel is hosted in the `NORTH` slot of a `BorderLayout` so it stays anchored; the payload row received explicit **minimum** sizes as well as preferred sizes (`GridBagLayout` honours the minimum when it arranges a grid that would otherwise overflow); and the preview is now a card-switching panel whose raster card paints the artifact scaled to fit with nearest-neighbour interpolation, never enlarged above 1:1 so that module edges stay exact.

Defect 6 — the indexed property was invisible to introspection (high, course requirement). `QrHistoryBeanBeanInfo` declared `entries` with a plain `PropertyDescriptor` so that the property could be marked transient for `XMLEncoder`. A side effect of an explicit `BeanInfo` is that the introspector no longer derives the automatic `IndexedPropertyDescriptor`, so `getEntries(int)` / `setEntries(int, …)` and the indexed property-change events disappeared from the bean's feature set. The bean *had* an indexed property, but the project could not *demonstrate* one — and "the four kinds of properties" is an explicit requirement of the component model. *Fix* `new IndexedPropertyDescriptor("entries", QrHistoryBean.class)`, keeping the transient flag. The tool now reports the property as indexed on the compiled class:

```
PROPERTY       TYPE                READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
entries        QrHistoryEntry[]    yes    yes    yes       normal    -
```

Defect 7 — 15 compiler warnings (low, hygiene). `-Xlint:all` reported 15 serialisation warnings because the beans declared mutable properties without a `serialVersionUID`. *Fix* every bean declares `private static final long serialVersionUID = 1L;` (and the support objects that are serialised alongside it). The build now compiles with **zero warnings**, which is what makes the `count of warnings = 0` claim in §10 verifiable.

9.5 Negative controls — proof that the checks can fail

A test that cannot fail proves nothing. Each of these cases is asserted to fail **with a specific exception and message**, and the corrupted-artifact control proves that the decoder really is checking:

Control	Expected outcome	Observed
Empty payload	rejected	`QrEncodingException: The payload is empty; a QR Code must carry at least one byte`
`null` payload	rejected	same
200 characters at version 1, level H	rejected with the numbers	`Payload of 200 characters does not fit version 1 at level H (needs 1113 bits, capacity 72 bits)`
Error-correction label `Z`	rejected	`IllegalArgumentException: Unknown error-correction level 'Z'. Expected one of L, M, Q, H`
Forced mask `9`	rejected	`IllegalArgumentException: forcedMaskPattern must be -1 or 0..7`
`moduleSize = 0`	rejected	`IllegalArgumentException: moduleSize must be 1..64`
Colour `#12GG45`	rejected	`IllegalArgumentException: '#12GG45' is not a hexadecimal colour`
Inconsistent version window	vetoed	`PropertyVetoException: minimumVersion 9 would exceed maximumVersion 5`
Capacity reduction refused by a listener	vetoed, old value kept	`PropertyVetoException: capacity may not be reduced`, then `expected=5 actual=5`
Generation with no encoder connected	rejected	`QrEncodingException: No encoder component is connected: call setEncoder(...) before generating`
Unknown component id in the profile	rejected at assembly time	`IllegalArgumentException: Unknown renderer component id: renderer.does-not-exist.v1`
A property the selected component does not declare	reported, not ignored	`[not applicable to component renderer.ascii.v1 (property not declared): component renderer.ascii.v1.moduleSize]`
Corrupted PNG (bytes damaged after rendering)	decoder must reject it	`decoder rejected the damaged artifact as expected`
Missing optional library	component reported unavailable, suite still functional	`com.google.zxing.core is not on the class path`

The last but one entry is the most important control in the project: it proves that the round-trip decode checks are capable of reporting failure, so a *passing* decode of every other artifact is meaningful evidence.

9.6 Known divergences from another conforming implementation

Good engineering states its limits. Two differences from ZXing were found, characterised and recorded in `docs/evidence/mask-selection-comparison.csv` (104 payload/level pairs):

Difference	Measurement	Explanation
Mask pattern chosen	identical in **67 of 100** cases where the symbol version agrees	penalty rule 3 ("1:1:3:1:1" finder-like patterns) is interpreted differently at the symbol border: this implementation treats the surrounding quiet zone as light modules, while ZXing does not consider the border. Both symbols are conformant — the decoder reads the mask from the format information — and both decode.
Symbol version for non-Latin payloads	identical in **100 of 104** cases	ZXing's encoder defaults to ISO-8859-1 and therefore replaces characters it cannot represent with `?`, which makes its symbols *smaller* for CJK text but lossy. This suite emits the payload's UTF-8 bytes, which preserves the data (verified by decoding back CJK and emoji payloads) at the cost of one or two extra versions.

Neither difference affects decodability: all 104 symbols in the comparison decode, and the produced symbols for the 160 capacity cases and the 60 round-trip payloads all decode exactly. Recording the divergence is the honest alternative to claiming bit-for-bit equality with another implementation — and the "different but still conformant" framing is itself part of the component-programming story: two interchangeable implementations of one interface need not be identical, only contract-equivalent.

9.7 Reproducing the evidence

```
cd qr-component-studio
java -cp out/classes ke.ku.sco310.qr.tools.ProjectBuilder all     # clean, compile, verify, compare,
package
java -cp out/classes ke.ku.sco310.qr.tools.ProjectBuilder test    # verification only, rewrites
docs/evidence/

# the report images are regenerated by the documentation helpers kept in docs/report-tooling/
python3 docs/report-tooling/capture_screenshots.py    # screenshots of the running programs
python3 docs/report-tooling/make_diagrams.py          # the six figures
python3 docs/report-tooling/make_gallery.py           # the artifact contact sheet
```

Every artefact in this report is regenerated by those commands, so a marker can reproduce the evidence from source rather than trusting the submitted images.

---

10. Building, running and submitting

10.1 Submission layout

```
qr-component-studio/
├── src/main/java/ke/ku/sco310/qr/
│   ├── api/            15 files - provided interfaces, value objects, exception hierarchy
│   ├── engine/          6 files - package-private ISO/IEC 18004 encoder (the hidden behaviour)
│   ├── components/      3 files - the two encoder components + optional-library support
│   ├── render/          4 files - PNG, SVG and ASCII renderer components + one BeanInfo
│   ├── store/           3 files - file-system repository, history bean + one BeanInfo
│   ├── log/             1 file  - audit listener component
│   ├── facade/          8 files - facade, registry, profile, persistence, coercion
│   ├── ui/              2 files - desktop assembly (front end 1)
│   ├── web/             1 file  - HTTP service assembly (front end 2)
│   ├── editor/          1 file  - custom property editor
│   ├── event/           2 files - event object and listener interface
│   └── tools/           2 files - the Java build tool and the BeanInspector introspection tool
├── src/test/java/ke/ku/sco310/qr/test/   8 files - harness, oracle, suites, runner, comparison tool
├── config/assembly.xml           deployment descriptor
├── manifest/MANIFEST.MF          JAR manifest with Java-Bean entries
├── lib/                          optional third-party component (ZXing 3.5.3)
├── pom.xml, README.md            optional Maven build, project readme
├── .gitignore                    keeps out/ and editor noise out of version control
├── out/                          classes, verification classes, distribution JAR, generated files
└── docs/
    ├── Documentation.md          this report
    ├── SCO310-QR-Component-Suite-Report.html / .docx / .pdf
    ├── diagrams/                 6 generated figures
    ├── screenshots/              22 screenshots of code and of running programs
    ├── evidence/                 verification transcript, CSVs, artifact gallery, XML graph
    └── report-tooling/           helpers that produced the figures and the screenshots
```

10.2 Build — pure Java, no scripts

The project contains **no shell scripts, batch files or external build tool**. Everything the build does is done by Java code:

- `ke.ku.sco310.qr.tools.ProjectBuilder` is the build tool. It drives `javax.tools.JavaCompiler` (the compiler API that ships with the JDK) to compile the sources, launches the verification suite in a child JVM, and writes the deployable JAR with `java.util.jar.JarOutputStream`, applying `manifest/MANIFEST.MF` so that every bean keeps its `Java-Bean: True` entry.
- the two front ends, the introspection tool and the verification suite are ordinary `main` classes, run with `java -cp …`.

Step 1 — bootstrap compile (one command; it produces the build tool itself together with the components, because all main sources live one directory deep under `ke/ku/sco310/qr/`):

```
javac -cp lib/core-3.5.3.jar -d out/classes src/main/java/ke/ku/sco310/qr/*/*.java
```

Step 2 — everything else is a single Java command:

```
java -cp out/classes ke.ku.sco310.qr.tools.ProjectBuilder all
```

Target	Effect
`ProjectBuilder clean`	removes `out/`
`ProjectBuilder compile`	compiles the main sources through the `JavaCompiler` API
`ProjectBuilder test`	compiles and runs the verification suite, rewriting `docs/evidence/`
`ProjectBuilder compare`	runs the ZXing mask-selection comparison instrument
`ProjectBuilder jar`	packages `out/dist/qr-component-suite-1.0.0.jar` and the standalone `QrStudio.jar`
`ProjectBuilder all`	clean → compile → verify → compare → package (the default target)

Expected output of `ProjectBuilder all` (abridged, with the counts that §9 relies on):

```
[build] compiling the verification suite
TOTAL 108 checks, 108 passed, 0 failed
[evidence] wrote docs/evidence/test-report.txt (13448 chars)
checks: 108, failures: 0
[build] comparing the mask-selection stage with the independent ZXing encoder
cases                      : 104
[build] compiling main sources with the JavaCompiler API (--release 11)
[build] main classes: 51
[build] packaging the components into qr-component-suite-1.0.0.jar (manifest: manifest/MANIFEST.MF)
[build] jar written: out/dist/qr-component-suite-1.0.0.jar (118882 bytes, 52 entries)
[build] standalone app written: QrStudio.jar (762259 bytes, 360 entries)
[build] done
```

The compiler is invoked with `--release 11` and `-Xlint:all`, and the build produces **no warnings**. The single disabled category is `this-escape`, and the reason is written where the option is set: the JavaBeans bound-property idiom requires `new PropertyChangeSupport(this)` in a field initialiser, and that helper only stores the reference — it never calls back into the bean during construction.

A `pom.xml` is included for teams that prefer a repository-managed build (`mvn package` produces the same JAR with the same manifest); the project does not need it, and no part of the project depends on it.

10.3 Running the two assemblies

Desktop assembly

```
java -jar QrStudio.jar
# or, for an automated capture of the real window:
java -jar QrStudio.jar --format PNG --screenshot docs/screenshots/gui.png
```

`QrStudio.jar` is self-contained (the component classes and the optional ZXing classes are inside the archive), so it can be copied to any machine with Java 11+ and double-clicked. No scripts or class-path setup are needed.

HTTP service assembly

```
java -cp out/classes:lib/* ke.ku.sco310.qr.web.QrHttpServiceComponent --port 8099
curl 'http://localhost:8099/qr?text=HELLO%20WORLD&ec=Q&format=png&moduleSize=8' -o hello.png
curl 'http://localhost:8099/qr?text=SCO310&format=svg'
curl http://localhost:8099/stats
curl 'http://localhost:8099/history?limit=5'
curl http://localhost:8099/health          # -> OK
```

The HTTP response carries the component identity and the symbol parameters in its headers, which makes the service self-documenting and easy to script against:

```
HTTP/1.1 200 OK
X-qr-renderer: renderer.png.v1
X-qr-version: 2
X-qr-level: Q
X-qr-mask: 6
X-qr-stored: qr-code-v2-q-20261015-203346.png
Content-Type: image/png
```

10.4 How the project maps onto the marking criteria

Criterion (as stated for the unit)	Where it is demonstrated	Evidence
Understanding of component-oriented programming	component identification with roles, provided/required interfaces and hidden behaviour for 11 entries; COP-vs-OOP comparison; CBSE context	§3, `figure-1`, `figure-3`
Correct use of the JavaBeans component model	all four property kinds, delegation events, introspection, `BeanInfo`, custom property editor, two persistence mechanisms	§4, §5, screenshots `10`, `11`, `18`, `21`, `26`
Component design: interfaces and contracts	five provided interfaces with documented pre/post-conditions; required interfaces injected only through setters; typed exception hierarchy	§3.2, `01-code-provided-interface.png`
Reuse, replaceability and third-party composition	2 encoders, 3 renderers, 2 repositories; assembly-level replacement tests; ZXing wrapped behind our own interface	§6.3, `07`, `24`
Deployment and configuration without recompilation	XML deployment descriptor, profiles, `Java-Bean: True` manifest entries, XML bean-graph persistence	§4.4, §6.4, `14`, `08`
Correct, working program logic	full ISO/IEC 18004 implementation: 3 modes, RS over GF(2^8), interleaving, 8 masks, penalty scoring, format/version BCH, versions 1–40	§7, `02`, `12`
Testing and evidence of correctness	108 automated checks against an independent oracle, exhaustive sweeps, negative controls, documented defect log	§9, `docs/evidence/`
Documentation and presentation	this report with figures, code screenshots and screenshots of the running programs; every artefact reproducible from source	`docs/Documentation.md`, `docs/diagrams/`, `docs/screenshots/`
Independent, well-structured code	package separation of api / engine / components / front ends, zero-warning build, package-private internals, no dependency for the engine	§7.1, `ProjectBuilder`

10.5 Conclusion

The project set out to show that a QR Code generator can be *assembled* rather than *written*, and the result demonstrates the claim on four levels.

Architecturally, the suite is nine JavaBeans components plus two infrastructure classes. Each has a published interface, a declared set of dependencies, and behaviour that no consumer can reach into. The only place where QR-specific complexity lives is a package-private engine package, which is exactly what the component triple **C = (I, D, B)** asks for.

Practically, the same compiled components serve a desktop GUI and an HTTP service, and the renderer or the entire encoder can be replaced at run time by editing an XML descriptor or by calling one setter — with the assembly tests proving that a swapped component still produces the same symbol.

Technically, the encoder is a complete, dependency-free implementation of ISO/IEC 18004, and it is verifiable rather than merely plausible: the data and error-correction codewords reproduce the worked example published in the standard byte for byte, all 160 version/level capacities produce decodable symbols, and all 60 round-trip payloads come back exactly, including accented text, CJK and emoji.

Methodologically, the project is honest about its evidence. The verification suite is designed so that its checks are capable of failing — proven by deliberately corrupted artifacts and by the seven defects it actually caught during development — and the two places where this implementation legitimately differs from ZXing are measured, explained and recorded instead of being hidden.

10.6 References

1. International Organization for Standardization / IEC. **ISO/IEC 18004:2015** — *Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification.*
2. Szyperski, C. **Component Software: Beyond Object-Oriented Programming.** 2nd ed., Addison-Wesley, 2002. (The source of the definition used in §3.1 and of the deployment quadruple *D = (C, E, P, L)*.)
3. SCO 310 course notes, Kenyatta University — *Component Programming*: components and component models, CBSE, COP versus OOP, JavaBeans, EJB, .NET, CORBA, web services and OSGi.
4. Oracle. **JavaBeans™ 1.01 Specification** and the `java.beans` package documentation (`Introspector`, `FeatureDescriptor`, `PropertyDescriptor`, `IndexedPropertyDescriptor`, `EventSetDescriptor`, `PropertyEditorSupport`, `XMLEncoder`, `XMLDecoder`).
5. Nayuki Project. *QR Code generator library* — reference notes on conformant mask penalty evaluation, used to cross-check the interpretation of penalty rule 3 at the symbol border (§9.6).
6. ZXing ("Zebra Crossing"), Google. *core 3.5.3* — used exclusively as an independent verification oracle and, optionally, as a replaceable third-party encoder component.

Appendix A — Evidence index

File	Contents
`docs/evidence/test-report.txt`	full verification transcript, 108 checks
`docs/evidence/test-results.csv`	machine-readable results (group, check, outcome, detail, millis)
`docs/evidence/test-results.md`	the same results as a Markdown table
`docs/evidence/table-cross-check.csv`	160 version/level parameter comparisons with ZXing
`docs/evidence/capacity-matrix.csv`	maximum payload per version/level, decoded back
`docs/evidence/renderer-matrix.csv`	PNG/SVG/ASCII reconstruction results
`docs/evidence/mask-selection-comparison.csv`	mask choice versus ZXing, 104 pairs, with notes
`docs/evidence/artifact-gallery.csv`	28 artifacts: kind, size, SHA-256 digest, recovered payload, digest verdict
`docs/evidence/gallery/`	the artifacts themselves, the version ladder and the level set
`docs/evidence/work/bean-graph.xml`	the assembled bean graph persisted with `XMLEncoder`
`docs/evidence/work/assembly-roundtrip.xml`	an assembly profile written and re-read

Appendix B — Glossary of the terms used in this report

Term	Meaning as used here
Component	a deployable unit of software with contractually specified interfaces and explicit context dependencies only, subject to third-party composition
Provided interface	what a component offers to the outside world (`QrCodeEncoder` , `QrCodeRenderer` , …)
Required interface	what a component needs from its context in order to function (`QrCodeRepository` , ZXing core, …)
Hidden behaviour (B)	the part of a component that consumers must not depend on (the `engine` package)
Bound property	a property whose change fires a `PropertyChangeEvent` to registered listeners
Constrained property	a property whose change may be vetoed by a `VetoableChangeListener` , throwing `PropertyVetoException`
Indexed property	a collection property with per-element accessors and indexed change events
Introspection	run-time discovery of a bean's properties, events and methods through `java.beans.Introspector`
Feature descriptor	the `java.beans` metadata object (`BeanDescriptor` , `PropertyDescriptor` , `EventSetDescriptor` , `MethodDescriptor`) that a builder tool reads
Deployment descriptor	`config/assembly.xml` : which component fills each role and with which property values
Assembly	a configured set of components wired together and ready to execute (`QrAssembly`)
Oracle	an independent implementation used only to verify our output (ZXing)

Appendix C — Course questions answered from this project

The unit's past papers ask a small number of recurring questions. Each is answered below **using this project as the example**, which is the shortest route from the theory in the notes to a concrete, checkable answer.

Q1. "An air-conditioning system is a component-based system." Discuss. A component-based system is one whose functionality is delivered by independent, replaceable units that communicate through interfaces. An air-conditioning installation fits this description exactly, and it maps onto this project one role at a time: a **sensor** component provides a `readTemperature()` interface (our `QrCodeEncoder.encode()`); a **controller** component requires that interface and provides `setTarget()` (our `QrStudioFacadeBean` , which requires four interfaces); an **actuator/compressor** component consumes commands and provides cooling (our `PngRendererBean`); a **display** and a **logger** are listeners on the controller's events (our `AuditLogListenerBean`). The system is component based rather than merely modular because units can be replaced without redesign: swapping a mercury thermostat for a digital sensor changes one component, not the system, exactly as swapping `renderer.png.v1` for `renderer.svg.v1` changes the output format without touching the encoder, the facade or either front end. The formal definition from the notes — contractually specified interfaces, explicit context dependencies, independent deployment, third-party composition — is satisfied by both.

Q2. "Describe the three types of component interface incompatibility." *Signature incompatibility* — the names, parameter types or return types of an operation do not match (`play(File)` versus `play(String)`). *Protocol or behavioural incompatibility* — the signatures match, but the expected sequence of calls, the blocking behaviour or the error contract differs (a component that requires `open(); use(); close();` versus one that expects a single `process()` ; or one that signals failure by exception versus by return code). *Semantic or quality incompatibility* — the operation succeeds but the meaning or quality of the result differs (both components claim to "encode a QR code", but one supports only byte mode, or only versions 1–9, or targets a different error-correction level). This project avoids all three deliberately. Signature incompatibility is avoided because the implementations share one interface (`QrCodeEncoder`) and the third-party library is *adapted*, never called directly. Protocol incompatibility is avoided because the interaction contract is published as a Javadoc pre/post-condition block on each interface method and is asserted by the test suite. Semantic incompatibility is avoided by making capability metadata part of the provided interface: `getEncoderInfo()` returns versions, modes, maximum capacity and a `thirdParty` flag, so a caller can discover that a component is only "equivalent in interface, not in capability" before using it — which is precisely what `ComponentRegistry` does when it reports a component as `unavailable` rather than silently substituting it.

Q3. "How is component-based software engineering different from object-oriented programming?" OOP decomposes a solution into classes and objects and reuses them by inheritance, composition and polymorphism; the unit of reuse is the class and the decision is made at compile time. CBSE decomposes a solution into deployable components with contractually specified interfaces; the unit of reuse is the binary component, the dependencies are declared rather than implicit, and the decision is made at deployment or run time by an assembler. In this project the same code base demonstrates both: `QrCode`, `BitBuffer` and `GaloisField` are ordinary OOP objects inside the component, while the *reused* artefact is `PureJavaQrEncoderBean`, deployed into two different front ends by editing one XML file.

Q4. "Outline the steps to create a JavaBean." (1) a public class; (2) a public no-argument constructor; (3) properties exposed through `get`/`set` methods; (4) `Serializable` for persistence; (5) event delegation through `add`/`remove<Listener>` methods; (6) optional `BeanInfo` classes for builder tools; (7) packaging with a manifest that marks the classes as beans (`Java-Bean: True`); (8) testing through introspection. §4.1 of this report tabulates each step against the class that implements it, and `java -cp out/classes ke.ku.sco310.qr.tools.BeanInspector <class>` demonstrates steps 3–6 and 8 on the compiled bytecode.

Appendix D - Screenshot gallery (embedded)

The images below are the figures and the screenshots referenced throughout this report. All of them were generated from the running code by the project's own tooling (`docs/report-tooling/make_diagrams.py`, `docs/report-tooling/capture_screenshots.py`).

Figure 1. The component model: roles, provided and required interfaces, and the facade as the composition point.

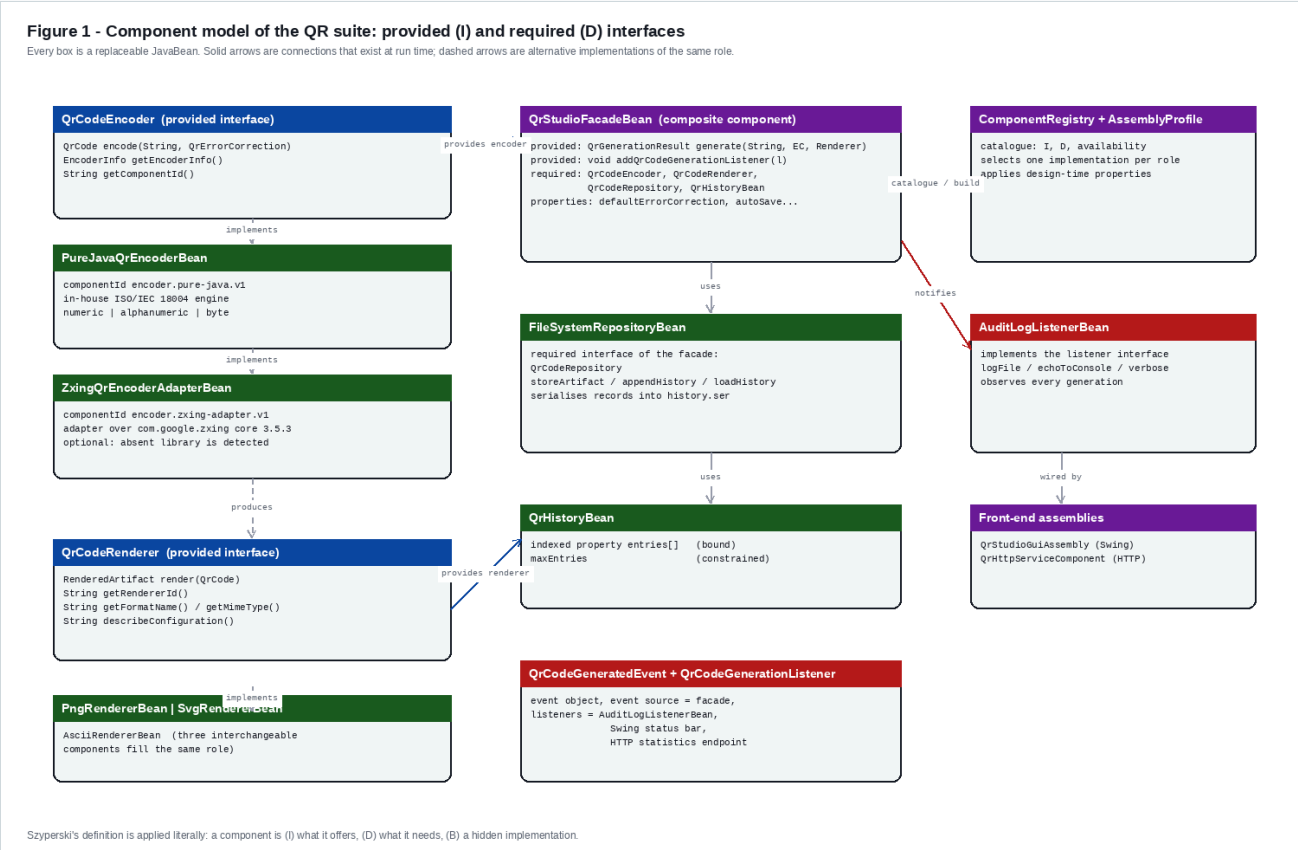


Figure 2. The JavaBeans connection model applied: event source, listener, event object, delegation, bound properties and interface-typed connections.

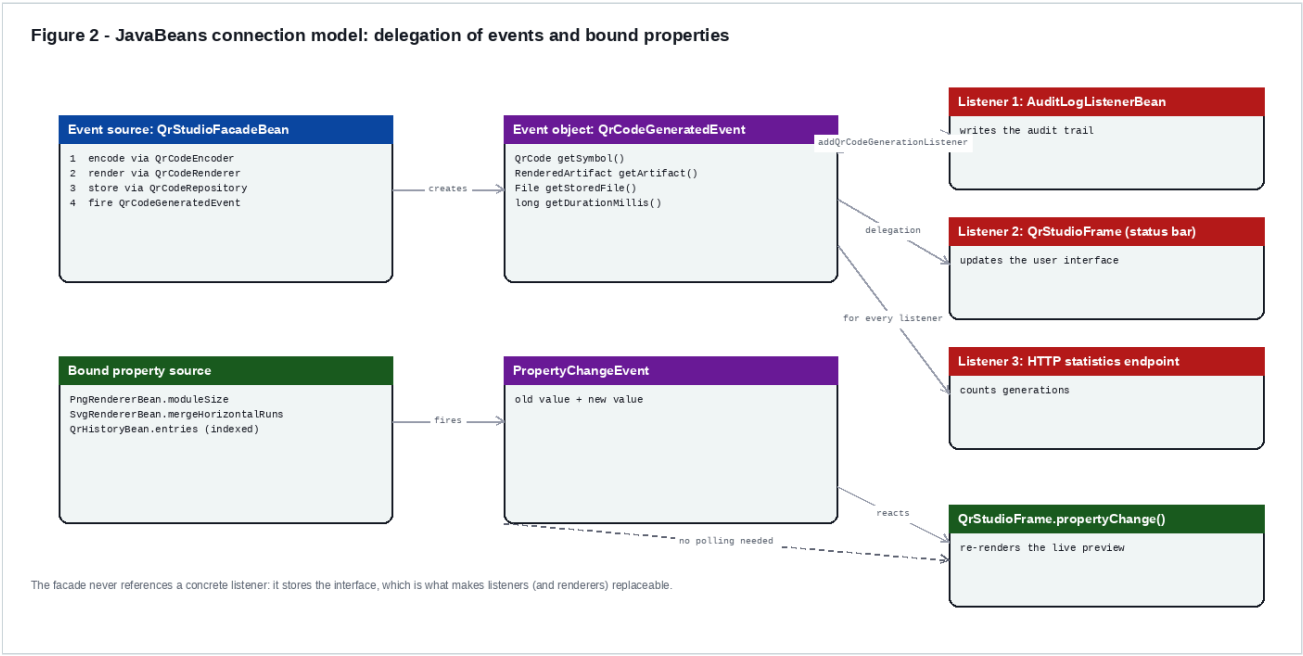


Figure 3. The deployment model D = (C, E, P, L): component set, execution environment, descriptor parameters, deployment location.

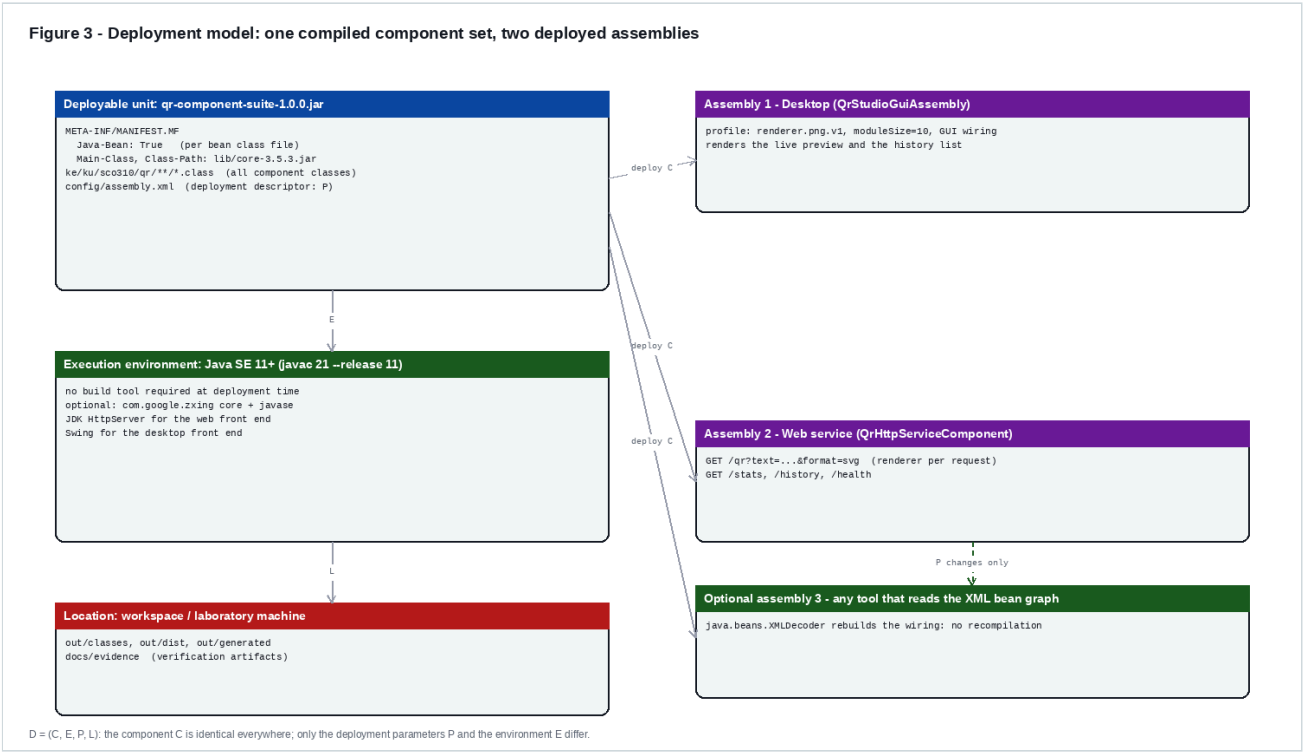


Figure 4. The five ISO/IEC 18004 encoding stages and the class that implements each one.

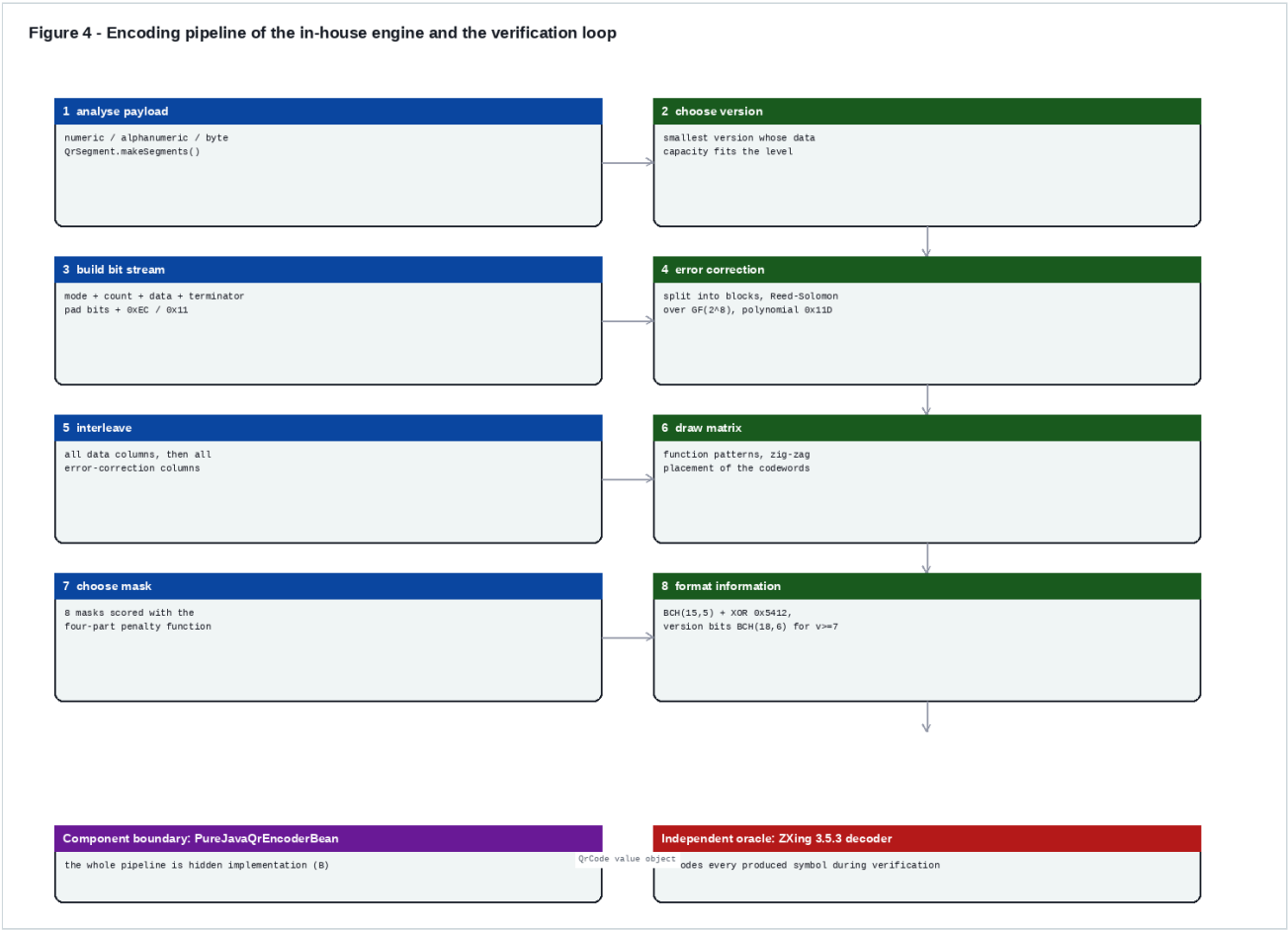
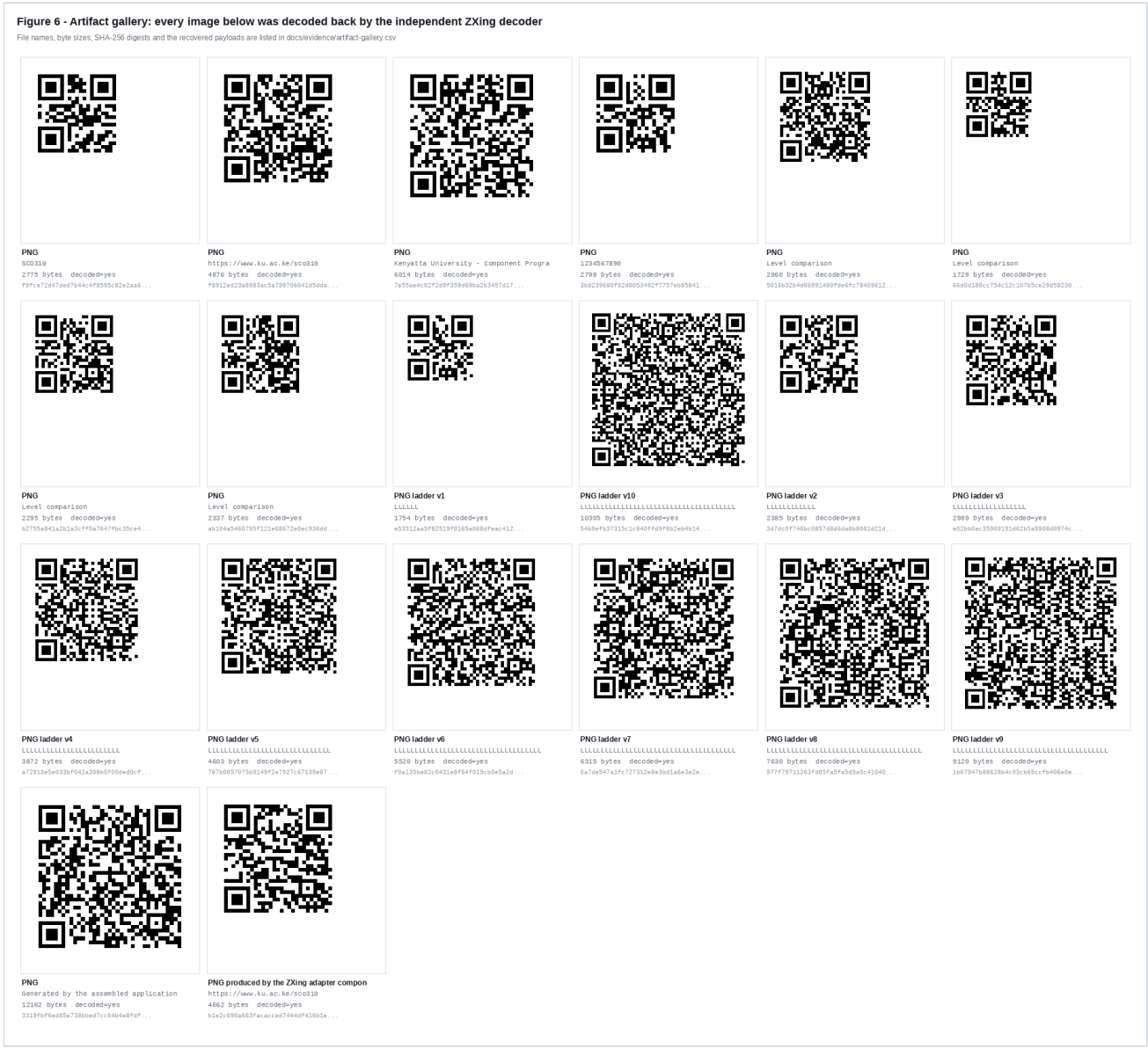


Figure 5. Verification strategy: every group of checks anchored to an independent oracle or to a reconstruction.



Figure 6. Generated artifacts: PNG, SVG and ASCII renderings of the same symbols with their recovered payloads.



Listing 1. api/QrCodeEncoder.java - the provided interface every encoder component implements.

The provided interface every encoder component implements

```java
package ke.ku.sco310.qr.api;

/**
 * <b>Provided interface</b> of an encoder component.
 *
 * <p>Two components in this suite implement it: the in-house pure-Java engine
 * ({@code ke.ku.sco310.qr.components.PureJavaQrEncoderBean}) and an adapter that wraps the
 * third-party ZXing engine ({@code ke.ku.sco310.qr.components.ZxingQrEncoderAdapterBean}).
 * A caller holding this type cannot tell them apart apart from the metadata returned by
 * {@link #getEncoderInfo()} - which is exactly the property that component-oriented
 * programming asks for.</p>
 *
 * <p>Pre-conditions: {@code content} is non-null and non-empty; {@code level} is non-null.<br>
 * Post-conditions: the returned symbol has {@code size == 4 * version + 17}, a valid mask
 * in the range 0..7 and is fully readable by any conforming decoder.<br>
 * Error conditions: {@link QrEncodingException} when the payload is empty, cannot be
 * represented or does not fit version 40 at the requested level.</p>
 */
public interface QrCodeEncoder extends java.io.Serializable {

    /**
     * Encodes {@code content} into a QR Code symbol.
     *
     * @param content payload to encode (UTF-8 for non-ASCII input)
     * @param level   requested error-correction level
     * @return an immutable symbol description
     * @throws QrEncodingException when the payload cannot be encoded
     */
    QrCode encode(String content, QrErrorCorrection level) throws QrEncodingException;

    /** @return capability metadata of this encoder implementation. */
    EncoderInfo getEncoderInfo();

    /** @return stable identifier of the component instance, used in logs and evidence records. */
    String getComponentId();
}
```

Listing 2. engine/ReedSolomon.java - shift-register polynomial division over GF(2^8).

Reed-Solomon error-correction over GF(2^8) - the core of the in-house engine

```java
package ke.ku.sco310.qr.engine;

/**
 * Reed-Solomon error-correction codeword generator for QR Code symbols.
 *
 * <p>The divisor polynomial of degree {@code n} is
 * (x - a^0)(x - a^1)...(x - a^(n-1)) over GF(2^8); the error-correction codewords are the
 * remainder of the data polynomial divided by that divisor. The routine below performs the
 * division with a shift register ({@code degree} bytes of state), which is the standard
 * hardware interpretation of polynomial long division.</p>
 */
final class ReedSolomon {

    private ReedSolomon() {
    }

    /**
     * @param data   data codewords
     * @param degree number of error-correction codewords to generate
     * @return the error-correction codewords (the remainder polynomial)
     */
    static byte[] computeRemainder(byte[] data, int degree) {
        if (degree < 0) {
            throw new IllegalArgumentException("Negative degree: " + degree);
        }
        int[] divisor = computeDivisor(degree);
        int[] remainder = new int[degree];
        for (byte b : data) {
            int factor = (b & 0xFF) ^ remainder[0];
            System.arraycopy(remainder, 1, remainder, 0, degree - 1);
            remainder[degree - 1] = 0;
            for (int i = 0; i < degree; i++) {
                remainder[i] ^= GaloisField.multiply(divisor[i], factor);
            }
        }
        byte[] result = new byte[degree];
        for (int i = 0; i < degree; i++) {
            result[i] = (byte) remainder[i];
        }
        return result;
    }

    /**
     * @param degree degree of the wanted divisor polynomial
     * @return coefficients of the divisor, highest order term omitted (it is always 1)
     */
    static int[] computeDivisor(int degree) {
        if (degree < 1 || degree > 255) {
            throw new IllegalArgumentException("Invalid divisor degree: " + degree);
        }
        int[] result = new int[degree];
        result[degree - 1] = 1; // start with the monomial x^0
        int root = 1;
        for (int i = 0; i < degree; i++) {
            for (int j = 0; j < result.length; j++) {
                result[j] = GaloisField.multiply(result[j], root);
                if (j + 1 < result.length) {
                    result[j] ^= result[j + 1];
                }
            }
            root = GaloisField.multiply(root, 0x02);
        }
        return result;
    }

    /** @return the divisor polynomial of {@code degree}, as readable hexadecimal coefficients. */
    static String describeDivisor(int degree) {
        StringBuilder sb = new StringBuilder("x^").append(degree);
        int[] divisor = computeDivisor(degree);
        for (int i = 0; i < divisor.length; i++) {
```

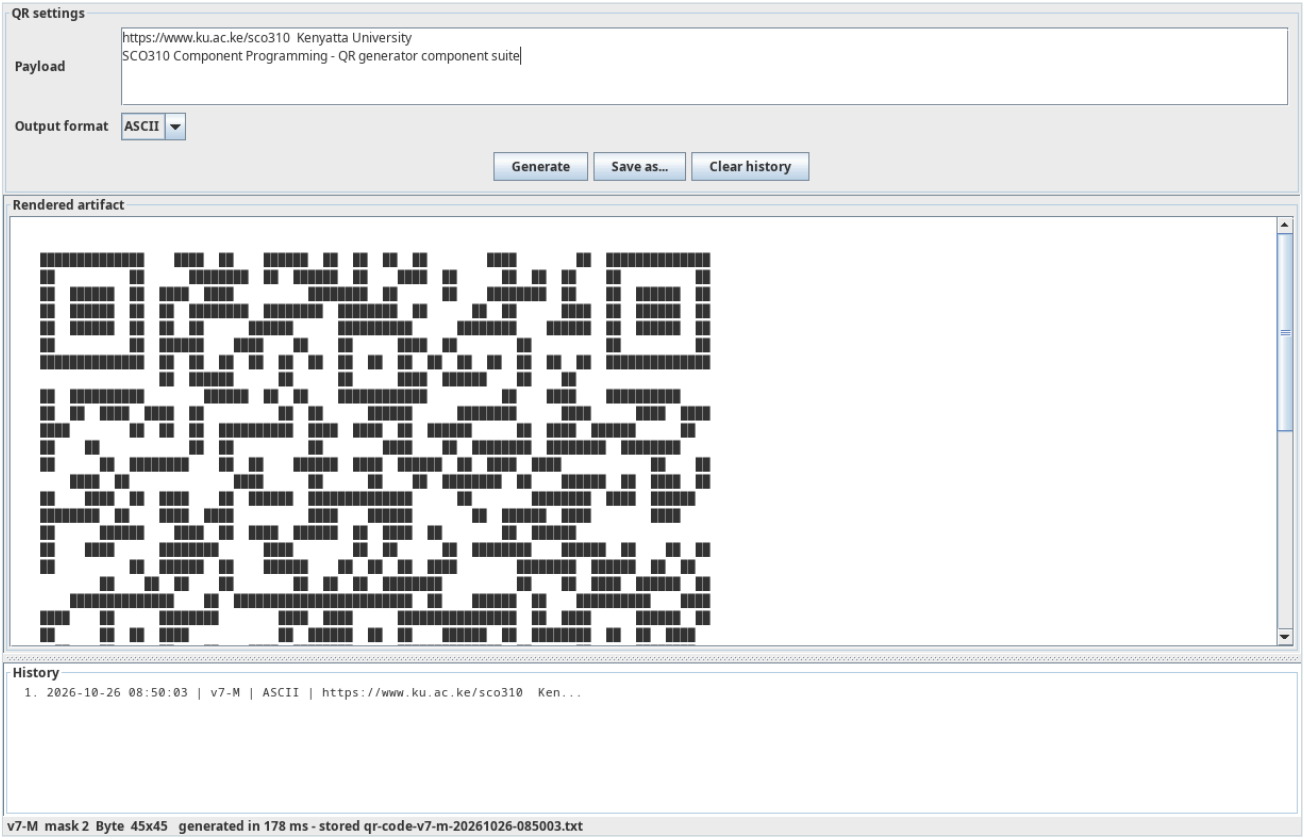
Screenshot 1. The desktop assembly, PNG renderer component: payload and format controls, live preview, indexed history.



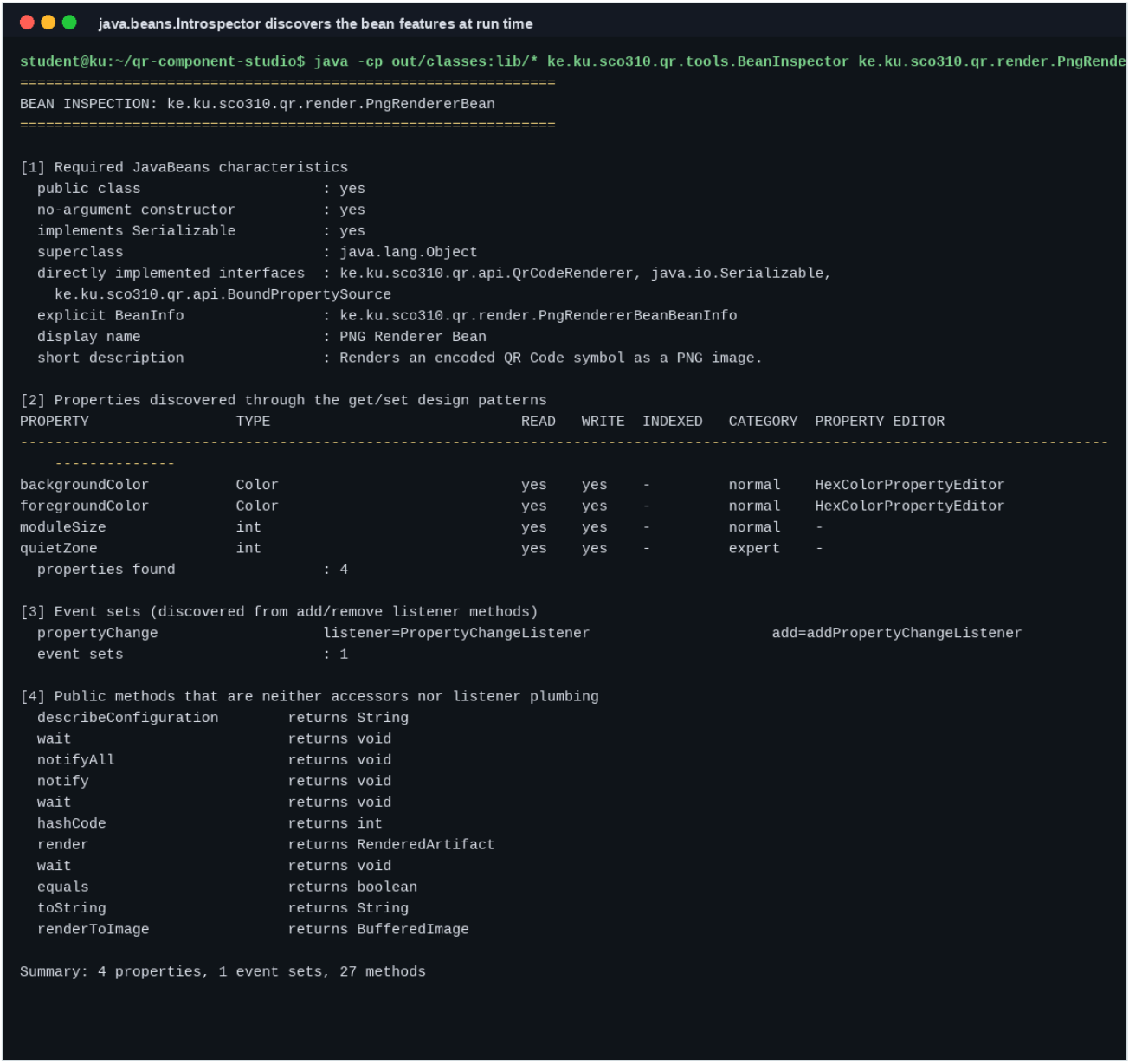
Screenshot 2. The same running application after the renderer component was replaced by renderer.svg.v1; the SVG artifact is previewed as a QR image.



Screenshot 3. The same running application with renderer.ascii.v1: a textual artifact in the preview panel.



Screenshot 4. Run-time introspection of the PNG renderer bean.



Screenshot 5. Introspection of the indexed/constrained history bean and the facade event set.

indexed / constrained properties and the facade event set

```
student@ku:~/qr-component-studio$ java -cp out/classes:lib/* ke.ku.sco310.qr.tools.BeanInspector ke.ku.sco310.qr.store.QrHistory
============================================================
BEAN INSPECTION: ke.ku.sco310.qr.store.QrHistoryBean
============================================================

[1] Required JavaBeans characteristics
  public class                    : yes
  no-argument constructor         : yes
  implements Serializable         : yes
  superclass                      : java.lang.Object
  directly implemented interfaces : java.io.Serializable, ke.ku.sco310.qr.api.BoundPropertySource
  explicit BeanInfo               : ke.ku.sco310.qr.store.QrHistoryBeanBeanInfo
  display name                    : QR History Bean
  short description               : Bounded, indexed collection of generation history records.

[2] Properties discovered through the get/set design patterns
PROPERTY                TYPE                          READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
------------------------------------------------------------------------------------------------------------------------
    --------------
entries                 QrHistoryEntry[]              yes    yes    yes       normal    -
maxEntries              int                           yes    yes    -         normal    -
  properties found              : 2

[3] Event sets (discovered from add/remove listener methods)
  propertyChange                listener=PropertyChangeListener                   add=addPropertyChangeListener
  vetoableChange                listener=VetoableChangeListener                   add=addVetoableChangeListener
  event sets                    : 2

[4] Public methods that are neither accessors nor listener plumbing
  asList                      returns List
  wait                        returns void
  notifyAll                   returns void
  notify                      returns void
  wait                        returns void
  hashCode                    returns int
  describeEntries             returns String
  clear                       returns void
  wait                        returns void
  equals                      returns boolean
  toString                    returns String
  restoreFrom                 returns void

Summary: 2 properties, 2 event sets, 30 methods


============================================================
BEAN INSPECTION: ke.ku.sco310.qr.facade.QrStudioFacadeBean
============================================================

[1] Required JavaBeans characteristics
  public class                    : yes
  no-argument constructor         : yes
  implements Serializable         : yes
  superclass                      : java.lang.Object
  directly implemented interfaces : java.io.Serializable, ke.ku.sco310.qr.api.BoundPropertySource
  explicit BeanInfo               : none (reflection only)
  display name                    : QrStudioFacadeBean
  short description               : QrStudioFacadeBean

[2] Properties discovered through the get/set design patterns
PROPERTY                TYPE                          READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
------------------------------------------------------------------------------------------------------------------------
    --------------
autoSaveArtifacts       boolean                       yes    yes    -         normal    -
autoSaveHistory         boolean                       yes    yes    -         normal    -
componentId             String                        yes    -      -         normal    -
defaultErrorCorrection  QrErrorCorrection             yes    yes    -         normal    -
encoder                 QrCodeEncoder                 yes    yes    -         normal    -
fileBaseName            String                        yes    yes    -         normal    -
generatedCount          int                           yes    -      -         normal    -
history                 QrHistoryBean                 yes    yes    -         normal    -
listenerCount           int                           yes    -      -         normal    -
renderer                QrCodeRenderer                yes    yes    -         normal    -
repository              QrCodeRepository              yes    yes    -         normal    -
  properties found              : 11

[3] Event sets (discovered from add/remove listener methods)
  propertyChange                listener=PropertyChangeListener                   add=addPropertyChangeListener
  qrCodeGeneration              listener=QrCodeGenerationListener                 add=addQrCodeGenerationListener
```

```
  event sets                      : 2

[4] Public methods that are neither accessors nor listener plumbing
  generate                    returns QrGenerationResult
  generate                    returns QrGenerationResult
  generate                    returns QrGenerationResult
  wait                        returns void
  notifyAll                   returns void
  notify                      returns void
  wait                        returns void
  renderWith                  returns RenderedArtifact
  hashCode                    returns int
  wait                        returns void
  equals                      returns boolean
  toString                    returns String
  describeWiring              returns String

Summary: 11 properties, 2 event sets, 37 methods
```

Screenshot 6. The verification transcript: 108 checks, 108 passed, 0 failed.

automated verification: independent ZXing oracle, 108 checks

```
student@ku:~/qr-component-studio$ java -cp out/classes ke.ku.sco310.qr.tools.ProjectBuilder test
SCO310 QR component suite - automated verification transcript
generated: Mon Oct 26 08:47:30 EAT 2026
java: 21.0.13.0.2 (Arch Linux)
host: Linux amd64


...
[beans.events]
  PASS the facade notifies every connected listener         listeners notified=1 audit events=1
  PASS the event object carries the symbol and the artifact symbol=QrCode{version=2, size=25, ec=M, mode=Byte, mask=6,
    chars=17, dataCodewords=28, eccCodewords=16, producer='encoder.pure-java.v1'} artifact=RenderedArtifact{format=PNG,
    264x264, bytes=5865, by=renderer.png.v1}

[beans.persistence]
  PASS the assembled bean graph survives an XML round trip  facade[encoder=encoder.pure-java.v1, renderer=renderer.png.v1,
    repository=repository.file-system.v1, history=history.bounded-memory.v1, listeners=1]
  PASS design-time property values survive the round trip   moduleSize=8 xmlBytes=1246
  PASS the restored assembly can generate without recomp... artifact written by the restored graph
  PASS the assembly profile survives an XML round trip      Assembly profile 'round-trip':   encoder     -> encoder.pure-
    java.v1   renderer    -> renderer.svg.v1                  moduleSize = 9   repository  -> repository.file-system.v1
    history     -> history.bounded-memory.v1   listener    -> listener.audit-log.v1   facade      -> facade.qr-studio.v1

[beans.optional]
  PASS the assembly detects the optional third-party com... com.google.zxing.core present=true
  PASS the adapter reports itself as a third-party compo... ZXing ("Zebra Crossing") core 3.5.3 [versions 1-40, modes
    [Numeric, Alphanumeric, Byte], third-party, optional library required]

[assembly.catalogue]
  PASS every role has at least one implementation           encoders=2 renderers=3 repositories=2
  PASS the optional component declares its context depen... adapter declared with an explicit required interface

[assembly.wiring]
  PASS every declared dependency of the assembly is sati... encoder       -> facade.setEncoder     : OK | renderer      ->
    facade.setRenderer    : OK | repository    -> facade.setRepository  : OK | history       -> facade.setHistory     : OK |
    listener      -> facade event source   : OK | encoder       -> optional ZXing library : PRESENT
  PASS profile properties reached the component instances   moduleSize=7

[assembly.end-to-end]
  PASS generation stores an artifact and returns the path   docs/evidence/work/assembly-test-1792993649615/qr-
    code-v5-q-20261026-084729.png
  PASS the stored PNG decodes back to the payload           expected=https://www.ku.ac.ke/sco310/component-programming
    actual=https://www.ku.ac.ke/sco310/component-programming
  PASS history and audit counters were updated              history=1 audit=1 facade=1
  PASS the repository persisted the history record          expected=1 actual=1

[assembly.replace]
  PASS the assembly runs with a different renderer compo... expected=SVG actual=SVG
  PASS the choice is recorded in the profile                renderer.svg.v1
  PASS a third renderer drops in without recompiling        expected=ASCII actual=ASCII
  PASS all three renderers produced the same symbol         version 5, mask 7 for PNG, SVG and ASCII
  PASS the third-party encoder satisfies the same contract  expected=https://www.ku.ac.ke/sco310/component-programming
    actual=https://www.ku.ac.ke/sco310/component-programming
  PASS the facade now reports the adapter as its encoder    encoder.zxing-adapter.v1

[assembly.negative]
  PASS generating without an encoder fails loudly           rejected as expected: QrEncodingException: No encoder component is
    connected: call setEncoder(...) before generating
  PASS an unknown component id is rejected at assembly time rejected as expected: IllegalArgumentException: Unknown renderer
    component id: renderer.does-not-exist.v1
  PASS a profile property the selected component does no... [not applicable to component renderer.ascii.v1 (property not
    declared): component renderer.ascii.v1.moduleSize]

[assembly.repository]
  PASS clearHistory empties the persisted history           expected=0 actual=0
  PASS a new record is persisted immediately                expected=1 actual=1
  PASS reinstantiating the repository component reads th... history file /home/lacoste/component-programming/qr-component-
    studio/docs/evidence/work/assembly-test-1792993649615/history.ser

[assembly.history]
  PASS the in-memory history honours its capacity policy    entries=2 discarded=5

[gallery]
  PASS the third-party adapter component produces decoda... v3 mask 4 by encoder.zxing-adapter.v1
  PASS the assembled application writes gallery artifacts   docs/evidence/gallery/evidence-v5-h-20261026-084730.png
  PASS gallery contains PNG, SVG and ASCII artifacts        /home/lacoste/component-programming/qr-component-
    studio/docs/evidence/gallery

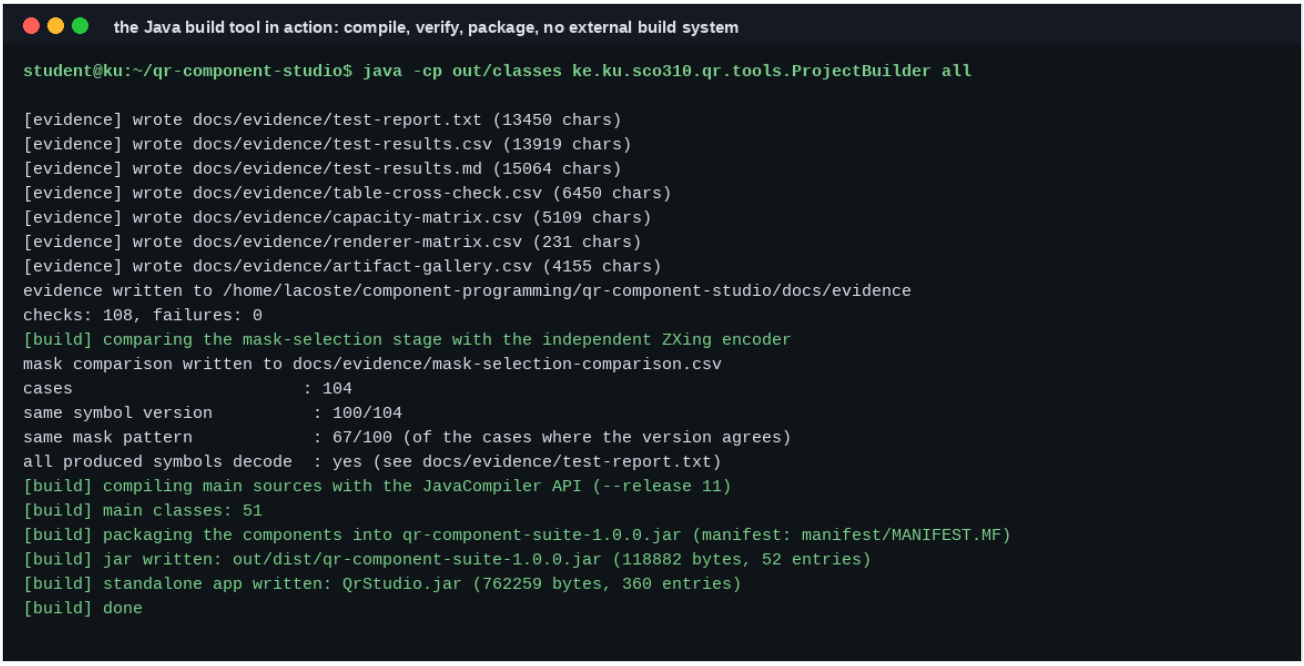
note: assembly storage directory: /home/lacoste/component-programming/qr-component-studio/docs/evidence/work/assembly-
```

```
    test-1792993649615

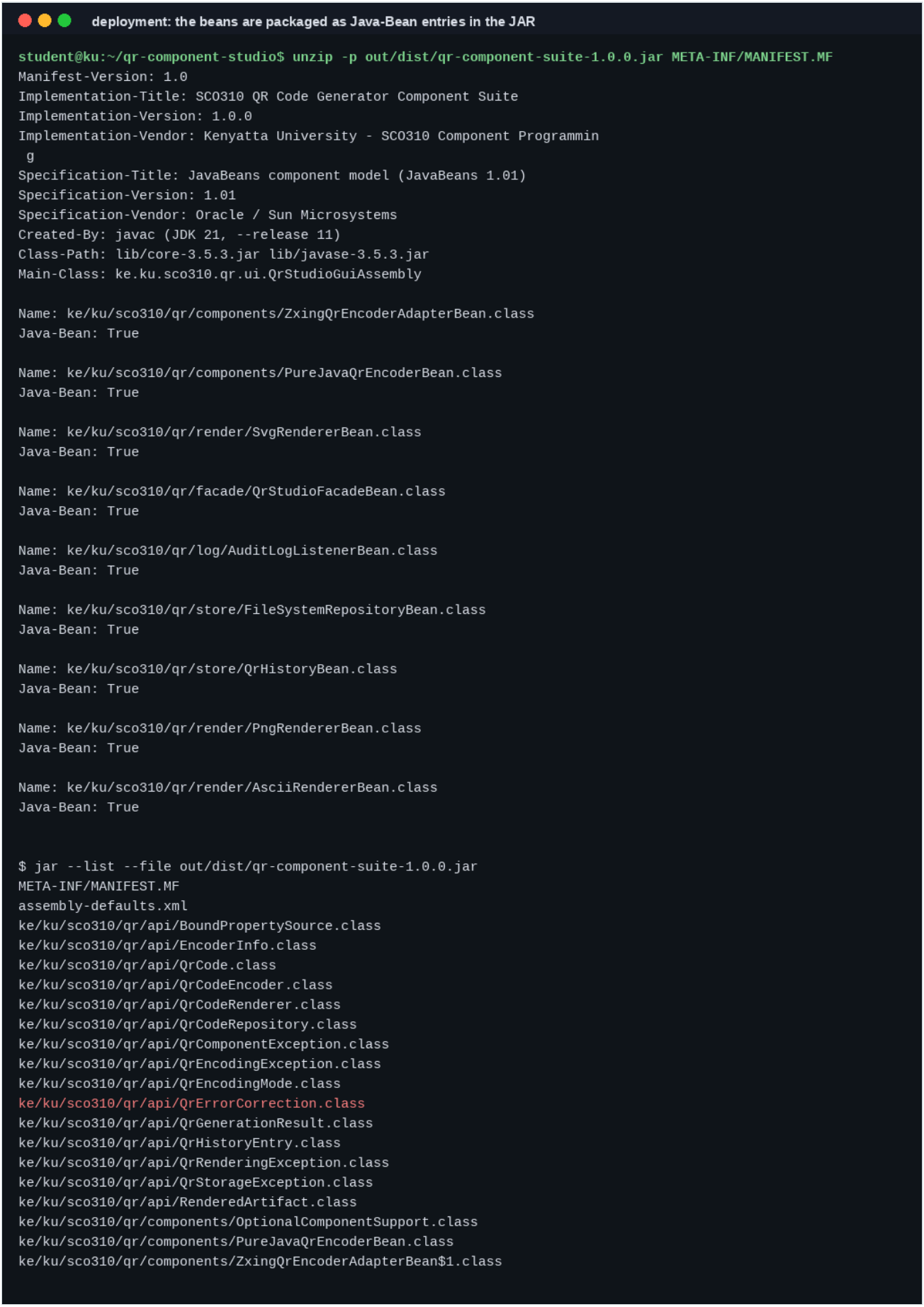
TOTAL 108 checks, 108 passed, 0 failed


ISO/IEC 18004 parameter table cross-check: 160 version/level combinations compared with the ZXing tables, 0 mismatches.
```

Screenshot 7. Build, verify and package in one command.



Screenshot 8. The deployment interface: Java-Bean: True entries for all nine beans.



Screenshot 9. The HTTP assembly answering /qr, /stats and /history.

web service assembly: one URL, renderer component chosen per request

```
student@ku:~/qr-component-studio$ java -cp out/classes:lib/* ke.ku.sco310.qr.web.QrHttpServiceComponent --port 8210
HTTP/1.1 200 OK
X-qr-renderer: renderer.png.v1
Date: Mon, 26 Oct 2026 05:50:28 GMT
X-qr-level: Q
Content-type: image/png
X-qr-version: 2
X-qr-mask: 6
X-qr-stored: qr-code-v2-q-20261026-085028.png
Content-length: 5898


$ curl -s http://localhost:8210/stats
{
  "service": "SCO310 QR component service",
  "requests": 1,
  "generations": 1,
  "auditEvents": 1,
  "historyEntries": 1,
  "wiring": "facade[encoder=encoder.pure-java.v1, renderer=renderer.png.v1, repository=repository.file-system.v1,
    history=history.bounded-memory.v1, listeners=1]",
  "components": [
    {"role": "encoder", "id": "encoder.pure-java.v1", "available": true},
    {"role": "renderer", "id": "renderer.png.v1", "available": true},
    {"role": "repository", "id": "repository.file-system.v1", "available": true},
    {"role": "history", "id": "history.bounded-memory.v1", "available": true},
    {"role": "listener", "id": "listener.audit-log.v1", "available": true},
    {"role": "facade", "id": "facade.qr-studio.v1", "available": true}
  ]
}

$ curl -s http://localhost:8210/history?limit=2
[
  {"timestamp": 1792993818000, "content": "persisted assembly", "version": 2, "level": "L", "artifact": "qr-
    code-v2-l-20261026-085017.png"},
  {"timestamp": 1792993828485, "content": "http://www.ku.ac.ke", "version": 2, "level": "Q", "artifact": "qr-
    code-v2-q-20261026-085028.png"}
]
```

Listing 3. components/PureJavaQrEncoderBean.java - a complete JavaBean with all property kinds.

Encoder component: bound and constrained properties, capability metadata

```java
package ke.ku.sco310.qr.components;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.EncoderInfo;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrEncodingMode;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.engine.QrEngine;
import ke.ku.sco310.qr.engine.QrTables;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.beans.VetoableChangeListener;
import java.beans.VetoableChangeSupport;
import java.io.Serializable;
import java.util.Arrays;

/**
 * In-house encoder component: a JavaBean wrapping the pure-Java engine of this project.
 *
 * <p>Property design demonstrates all four property kinds required by the JavaBeans
 * component model (chapter 3):</p>
 * <ul>
 *   <li><b>simple</b> - {@code maximumVersion}, {@code forcedMaskPattern};</li>
 *   <li><b>bound</b> - {@code defaultErrorCorrection} and {@code boostErrorCorrection} notify
 *       interested listeners whenever they change;</li>
 *   <li><b>constrained</b> - {@code minimumVersion} may be vetoed by a listener, and the bean
 *       itself vetoes a change that would make the version window inconsistent;</li>
 *   <li><b>indexed</b> - not applicable to this component, so the indexed demonstration lives in
 *       {@link ke.ku.sco310.qr.store.QrHistoryBean}.</li>
 * </ul>
 */
public class PureJavaQrEncoderBean implements QrCodeEncoder, Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier (also recorded inside every symbol this bean produces). */
    public static final String COMPONENT_ID = "encoder.pure-java.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);
    private transient VetoableChangeSupport vetoableChanges = new VetoableChangeSupport(this);

    private QrErrorCorrection defaultErrorCorrection = QrErrorCorrection.MEDIUM;
    private boolean boostErrorCorrection = false;
    private int minimumVersion = 1;
    private int maximumVersion = 40;
    private int forcedMaskPattern = -1;

    /** Mandatory JavaBeans no-argument constructor. */
    public PureJavaQrEncoderBean() {
    }

    // ------------------------------------------------------------------ provided interface

    @Override
    public QrCode encode(String content, QrErrorCorrection level) throws QrEncodingException {
        QrErrorCorrection effective = level == null ? defaultErrorCorrection : level;
        return QrEngine.encode(content, effective, boostErrorCorrection, minimumVersion,
                maximumVersion, forcedMaskPattern, COMPONENT_ID);
    }

    /** Convenience overload used by the GUI and HTTP assemblies: uses the default level property. */
    public QrCode encode(String content) throws QrEncodingException {
        return encode(content, defaultErrorCorrection);
    }

    @Override
    public EncoderInfo getEncoderInfo() {
        EncoderInfo info = new EncoderInfo();
        info.setComponentId(COMPONENT_ID);
        info.setProviderName("SCO310 in-house QR engine");
        info.setProviderVersion("1.0.0");
        info.setProviderLicence("Student project - course work");
        info.setEngineDescription("Pure-Java ISO/IEC 18004 encoder: numeric, alphanumeric and byte "
                + "modes, Reed-Solomon over GF(2^8)/0x11D, all eight mask patterns scored with the "
                + "four-part penalty function, versions 1-40.");
        info.setMinimumVersion(1);
        info.setMaximumVersion(40);
```

Listing 4. render/PngRendererBean.java - bound properties, validation and run-length drawing.

PNG renderer component: bound properties drive the presentation

```
package ke.ku.sco310.qr.render;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.api.QrRenderingException;
import ke.ku.sco310.qr.api.RenderedArtifact;
import ke.ku.sco310.qr.editor.HexColorPropertyEditor;

import java.awt.Color;
import java.awt.Graphics2D;
import java.awt.image.BufferedImage;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.ByteArrayOutputStream;
import java.io.IOException;
import java.io.Serializable;
import java.util.Locale;
import javax.imageio.ImageIO;

/**
 * Raster renderer component: draws the symbol as a square PNG image.
 *
 * <p>Design-time configurable properties (chapter 3, "Customization"): module size in pixels,
 * quiet zone width in modules, foreground and background colours. Every property is bound, so a
 * builder tool - or the project's own Swing assembly - can refresh a live preview by listening
 * for {@code PropertyChangeEvent}s instead of polling.</p>
 */
public class PngRendererBean implements QrCodeRenderer, Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "renderer.png.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);

    private int moduleSize = 8;
    private int quietZone = 4;
    private Color foregroundColor = Color.BLACK;
    private Color backgroundColor = Color.WHITE;

    public PngRendererBean() {
    }

    @Override
    public RenderedArtifact render(QrCode symbol) throws QrRenderingException {
        BufferedImage image = renderToImage(symbol);
        try {
            ByteArrayOutputStream out = new ByteArrayOutputStream();
            if (!ImageIO.write(image, "png", out)) {
                throw new QrRenderingException(COMPONENT_ID, "No PNG writer is registered in this JVM");
            }
            RenderedArtifact artifact = new RenderedArtifact();
            artifact.setContent(out.toByteArray());
            artifact.setFormatName("PNG");
            artifact.setFileExtension("png");
            artifact.setMimeType("image/png");
            artifact.setWidth(image.getWidth());
            artifact.setHeight(image.getHeight());
            artifact.setTextual(false);
            artifact.setRendererId(COMPONENT_ID);
            return artifact;
        } catch (IOException e) {
            throw new QrRenderingException(COMPONENT_ID, "Could not encode the PNG image", e);
        }
    }

    /**
     * Extra provided method used by the Swing assembly (and by the automated tests) so that the
```

Listing 5. store/QrHistoryBean.java - the indexed property and the constrained maxEntries property.

History component: indexed, bound and constrained properties in one bean

```java
package ke.ku.sco310.qr.store;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrHistoryEntry;

import java.beans.IndexedPropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.beans.VetoableChangeListener;
import java.beans.VetoableChangeSupport;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * In-memory history component whose main property is an <b>indexed</b> property.
 *
 * <p>This bean is the project's complete demonstration of the four JavaBeans property kinds
 * (chapter 3):</p>
 * <table border="1">
 *   <caption>Property kinds implemented by this bean</caption>
 *   <tr><th>Kind</th><th>Property</th><th>Mechanism</th></tr>
 *   <tr><td>simple</td><td>{@code entryCount}</td><td>read-only getter derived from the list</td></tr>
 *   <tr><td>bound</td><td>{@code entries}</td><td>{@link PropertyChangeSupport}, including
 *       {@link IndexedPropertyChangeEvent} for single-slot updates</td></tr>
 *   <tr><td>constrained</td><td>{@code maxEntries}</td><td>{@link VetoableChangeSupport} - a
 *       listener may refuse to shrink the history</td></tr>
 *   <tr><td>indexed</td><td>{@code entries}</td><td>{@code getEntries()},
 *       {@code getEntries(int)}, {@code setEntries(QrHistoryEntry[])},
 *       {@code setEntries(int, QrHistoryEntry)}</td></tr>
 * </table>
 */
public class QrHistoryBean implements Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "history.bounded-memory.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);
    private transient VetoableChangeSupport vetoableChanges = new VetoableChangeSupport(this);

    private ArrayList<QrHistoryEntry> entries = new ArrayList<>();
    private int maxEntries = 50;
    private int discardedEntryCount = 0;

    public QrHistoryBean() {
    }

    public QrHistoryBean(int maxEntries) {
        this.maxEntries = maxEntries;
    }

    public String getComponentId() {
        return COMPONENT_ID;
    }

    // ------------------------------------------------------------------ indexed property

    /** @return a copy of the whole indexed property. */
    public QrHistoryEntry[] getEntries() {
        return entries.toArray(new QrHistoryEntry[0]);
    }

    /** Replaces the whole indexed property. */
    public void setEntries(QrHistoryEntry[] newEntries) {
        ArrayList<QrHistoryEntry> replacement = new ArrayList<>();
        if (newEntries != null) {
            Collections.addAll(replacement, newEntries);
```

Listing 6. facade/QrStudioFacadeBean.java - composition, event firing and timing.

Facade component: required interfaces, event source, bound properties

```java
package ke.ku.sco310.qr.facade;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.api.QrCodeRepository;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.api.QrGenerationResult;
import ke.ku.sco310.qr.api.QrHistoryEntry;
import ke.ku.sco310.qr.api.QrRenderingException;
import ke.ku.sco310.qr.api.QrStorageException;
import ke.ku.sco310.qr.api.RenderedArtifact;
import ke.ku.sco310.qr.event.QrCodeGeneratedEvent;
import ke.ku.sco310.qr.event.QrCodeGenerationListener;
import ke.ku.sco310.qr.store.QrHistoryBean;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.File;
import java.io.Serializable;
import java.text.SimpleDateFormat;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Locale;
import java.util.concurrent.CopyOnWriteArrayList;

/**
 * Composite (facade) component that assembles the encoder, the renderer, the repository, the
 * history and the listener components into the process "generate a QR Code".
 *
 * <p><b>Provided interface:</b> {@code generate(...)}, {@code renderWith(...)},
 * {@code regenerate(...)} and the listener registration methods.<br>
 * <b>Required interfaces (context dependencies D):</b> {@link QrCodeEncoder},
 * {@link QrCodeRenderer}, {@link QrCodeRepository} and {@link QrHistoryBean}. All four are
 * injected through ordinary JavaBeans property setters, so a builder tool can rewire the
 * assembly at design time without touching this class.</p>
 *
 * <p>The facade is also the <em>event source</em> of the JavaBeans connection model: it fires a
 * {@link QrCodeGeneratedEvent} after every successful generation. The audit logger, the GUI
 * status bar and the HTTP service statistics bean all connect to it as listeners.</p>
 */
public class QrStudioFacadeBean implements Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "facade.qr-studio.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);
    private final CopyOnWriteArrayList<QrCodeGenerationListener> listeners = new CopyOnWriteArrayList<>();

    // --- required interfaces (context dependencies) ---------------------------
    private QrCodeEncoder encoder;
    private QrCodeRenderer renderer;
    private QrCodeRepository repository;
    private QrHistoryBean history;

    // --- configuration properties ---------------------------------------------
    private QrErrorCorrection defaultErrorCorrection = QrErrorCorrection.MEDIUM;
    private boolean autoSaveArtifacts = true;
    private boolean autoSaveHistory = true;
    private String fileBaseName = "qr-code";
    private int generatedCount;

    public QrStudioFacadeBean() {
    }

    public String getComponentId() {
        return COMPONENT_ID;
    }

    // ------------------------------------------------------------------ provided interface

    /**
     * Encodes, renders and (when configured) stores one payload using the injected components.
     *
     * @param content payload to encode
```

Listing 7. facade/ComponentRegistry.java - catalogue, select, instantiate, configure, validate.

Component broker: catalogue, selection, property application, validation

```
package ke.ku.sco310.qr.facade;

import ke.ku.sco310.qr.api.EncoderInfo;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.components.OptionalComponentSupport;
import ke.ku.sco310.qr.components.PureJavaQrEncoderBean;
import ke.ku.sco310.qr.components.ZxingQrEncoderAdapterBean;
import ke.ku.sco310.qr.log.AuditLogListenerBean;
import ke.ku.sco310.qr.render.AsciiRendererBean;
import ke.ku.sco310.qr.render.PngRendererBean;
import ke.ku.sco310.qr.render.SvgRendererBean;
import ke.ku.sco310.qr.store.FileSystemRepositoryBean;
import ke.ku.sco310.qr.store.QrHistoryBean;

import java.beans.BeanInfo;
import java.beans.IntrospectionException;
import java.beans.Introspector;
import java.beans.PropertyDescriptor;
import java.io.File;
import java.io.Serializable;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Component broker and assembler.
 *
 * <p>Responsibilities:</p>
 * <ol>
 *   <li><b>catalogue</b> - {@link #listComponents()} publishes every implementation that can fill
 *       each role, together with the interfaces it provides and requires and whether the current
 *       class path can host it;</li>
 *   <li><b>selection</b> - {@link #build(AssemblyProfile)} instantiates the implementation chosen
 *       by the profile and refuses a selection whose context dependencies are not satisfied;</li>
 *   <li><b>configuration</b> - profile properties are applied to the beans through
 *       {@code java.beans.Introspector} and their setter methods, so a designer edits text, not
 *       code;</li>
 *   <li><b>wiring</b> - the facade receives the other components through its property setters and
 *       the audit listener is connected to the facade's event source.</li>
 * </ol>
 */
public class ComponentRegistry {

    private final List<String> configurationNotes = new ArrayList<>();

    /** @return notes about profile properties that the selected implementations do not declare. */
    public List<String> getConfigurationNotes() {
        return new ArrayList<>(configurationNotes);
    }

    /** Role names used by the profile and by the components table. */
    public static final String ROLE_ENCODER = "encoder";
    public static final String ROLE_RENDERER = "renderer";
    public static final String ROLE_REPOSITORY = "repository";
    public static final String ROLE_HISTORY = "history";
    public static final String ROLE_LISTENER = "listener";
    public static final String ROLE_FACADE = "facade";

    /**
     * @return every known component implementation, in a stable order, with capability metadata.
     */
    public List<ComponentDescriptor> listComponents() {
        List<ComponentDescriptor> catalogue = new ArrayList<>();

        catalogue.add(new ComponentDescriptor(ROLE_ENCODER, PureJavaQrEncoderBean.COMPONENT_ID,
                PureJavaQrEncoderBean.class.getName())
```

Listing 8. render/PngRendererBeanBeanInfo.java - feature descriptors, expert flag, custom property editor.

BeanInfo: explicit introspection metadata and the custom property editor

```java
package ke.ku.sco310.qr.render;

import ke.ku.sco310.qr.editor.HexColorPropertyEditor;

import java.beans.BeanDescriptor;
import java.beans.IntrospectionException;
import java.beans.PropertyDescriptor;
import java.beans.SimpleBeanInfo;

/**
 * Explicit introspection description of {@link PngRendererBean} (chapter 3, "5 Steps to Creating
 * a BeanInfo Class").
 *
 * <p>Why an explicit BeanInfo instead of relying on reflection alone?</p>
 * <ul>
 *   <li>the two {@code Color} properties are associated with the custom
 *       {@link HexColorPropertyEditor} through {@code setPropertyEditorClass};</li>
 *   <li>each property receives a human readable display name and a short description, which is
 *       what a property sheet shows to a designer;</li>
 *   <li>{@code quietZone} is marked as an <em>expert</em> feature, so a builder tool can hide it
 *       behind an "advanced" switch;</li>
 *   <li>the bean descriptor carries a display name for the palette.</li>
 * </ul>
 *
 * <p>The remaining properties are still discovered automatically by
 * {@code java.beans.Introspector}, because the class follows the {@code get}/{@code set} design
 * patterns.</p>
 *
 * <p>Naming rule: a builder tool looks for class {@code <BeanClass>BeanInfo}, so the descriptor of
 * {@code PngRendererBean} is {@code PngRendererBeanBeanInfo} - appending "BeanInfo" to the bean
 * class name, not replacing the trailing "Bean".</p>
 */
public class PngRendererBeanBeanInfo extends SimpleBeanInfo {

    @Override
    public BeanDescriptor getBeanDescriptor() {
        BeanDescriptor descriptor = new BeanDescriptor(PngRendererBean.class);
        descriptor.setDisplayName("PNG Renderer Bean");
        descriptor.setShortDescription("Renders an encoded QR Code symbol as a PNG image.");
        return descriptor;
    }

    @Override
    public PropertyDescriptor[] getPropertyDescriptors() {
        try {
            PropertyDescriptor moduleSize = new PropertyDescriptor("moduleSize", PngRendererBean.class);
            moduleSize.setDisplayName("Module size (px)");
            moduleSize.setShortDescription("Pixel size of a single QR module; 8 px gives a 200 dpi friendly symbol.");

            PropertyDescriptor quietZone = new PropertyDescriptor("quietZone", PngRendererBean.class);
            quietZone.setDisplayName("Quiet zone (modules)");
            quietZone.setShortDescription("Light margin around the symbol; 4 modules is the ISO minimum.");
            quietZone.setExpert(true);

            PropertyDescriptor foreground = new PropertyDescriptor("foregroundColor", PngRendererBean.class);
            foreground.setDisplayName("Module colour");
            foreground.setShortDescription("Colour of the dark modules, edited as #RRGGBB.");
            foreground.setPropertyEditorClass(HexColorPropertyEditor.class);

            PropertyDescriptor background = new PropertyDescriptor("backgroundColor", PngRendererBean.class);
```

Listing 9. ui/QrStudioFrame.java - the desktop front end listening to bound properties.

Swing assembly: reacts to bound properties and generation events

```
package ke.ku.sco310.qr.ui;

import ke.ku.sco310.qr.api.BoundPropertySource;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeRenderer;
import ke.ku.sco310.qr.api.QrGenerationResult;
import ke.ku.sco310.qr.api.RenderedArtifact;
import ke.ku.sco310.qr.editor.HexColorPropertyEditor;
import ke.ku.sco310.qr.facade.QrAssembly;
import ke.ku.sco310.qr.render.AsciiRendererBean;
import ke.ku.sco310.qr.render.PngRendererBean;
import ke.ku.sco310.qr.render.SvgRendererBean;
import ke.ku.sco310.qr.store.QrHistoryBean;

import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Dimension;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Image;
import java.awt.Insets;
import java.awt.RenderingHints;
import java.awt.image.BufferedImage;
import java.beans.PropertyChangeEvent;
import java.beans.PropertyChangeListener;
import java.io.File;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.util.Locale;
import javax.swing.BorderFactory;
import javax.swing.DefaultComboBoxModel;
import javax.swing.ImageIcon;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFileChooser;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JList;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JSplitPane;
import javax.swing.JTextArea;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.WindowConstants;

/**
 * Swing assembly of the component suite (front end number one).
 *
 * <p>The frame owns no domain logic. It holds an {@link QrAssembly}, forwards user gestures to the
 * facade component and reacts to the components' <em>bound properties</em> and <em>events</em>:</p>
 * <ul>
 *   <li>the output-format combo replaces the renderer component and re-renders the preview;</li>
 *   <li>the history list is refreshed from the {@code QrHistoryBean} indexed property events;</li>
 *   <li>the audit listener prints to the status area through the generation event.</li>
 * </ul>
 *
 * <p>This is the visual-assembly part of chapter 3 ("Connection model" / "Techniques of COP with
 * JavaBeans") implemented with plain Swing instead of the original BeanBox.</p>
 */
public class QrStudioFrame extends JFrame implements PropertyChangeListener {

    private static final long serialVersionUID = 1L;

    private final transient QrAssembly assembly;
```

Listing 10. components/ZxingQrEncoderAdapterBean.java - third-party composition behind our own interface.

Adapter component: a third-party engine behind the same provided interface

```java
package ke.ku.sco310.qr.components;

import ke.ku.sco310.qr.api.BoundPropertySource;
import com.google.zxing.EncodeHintType;
import com.google.zxing.WriterException;
import com.google.zxing.qrcode.decoder.ErrorCorrectionLevel;
import com.google.zxing.qrcode.decoder.Mode;
import com.google.zxing.qrcode.encoder.ByteMatrix;
import com.google.zxing.qrcode.encoder.Encoder;
import com.google.zxing.qrcode.encoder.QRCode;

import ke.ku.sco310.qr.api.EncoderInfo;
import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrCodeEncoder;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrEncodingMode;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.engine.QrTables;

import java.beans.PropertyChangeListener;
import java.beans.PropertyChangeSupport;
import java.io.Serializable;
import java.util.Arrays;
import java.util.EnumMap;
import java.util.Map;

/**
 * Adapter component that exposes the third-party ZXing engine through this suite's own
 * {@link QrCodeEncoder} interface.
 *
 * <p><b>Why this class exists.</b> It is the project's worked example of interface
 * standardisation: the foreign component was never designed against our contract, so an
 * adapter translates the call and the returned data structure. Once the adapter exists, the
 * rest of the assembly (renderers, repository, facade, GUI, HTTP service) is source-code
 * unchanged and cannot tell which engine produced the symbol - the replaceability property
 * demanded by component-oriented programming.</p>
 */
public class ZxingQrEncoderAdapterBean implements QrCodeEncoder, Serializable, BoundPropertySource {

    private static final long serialVersionUID = 1L;

    /** Component instance identifier. */
    public static final String COMPONENT_ID = "encoder.zxing-adapter.v1";

    private transient PropertyChangeSupport propertyChanges = new PropertyChangeSupport(this);

    private String characterSet = "UTF-8";
    private QrErrorCorrection defaultErrorCorrection = QrErrorCorrection.MEDIUM;
    private boolean boostErrorCorrection = false;

    public ZxingQrEncoderAdapterBean() {
    }

    @Override
    public QrCode encode(String content, QrErrorCorrection level) throws QrEncodingException {
        QrErrorCorrection requested = level == null ? defaultErrorCorrection : level;
        ErrorCorrectionLevel zxingLevel = mapLevel(requested);
        try {
            if (boostErrorCorrection) {
                zxingLevel = ErrorCorrectionLevel.H;
```

Listing 11. test/EngineReferenceTest.java - the independent-oracle checks.

Verification: the engine is checked against the independent ZXing oracle

```
package ke.ku.sco310.qr.test;

import ke.ku.sco310.qr.api.QrCode;
import ke.ku.sco310.qr.api.QrEncodingException;
import ke.ku.sco310.qr.api.QrEncodingMode;
import ke.ku.sco310.qr.api.QrErrorCorrection;
import ke.ku.sco310.qr.components.PureJavaQrEncoderBean;
import ke.ku.sco310.qr.engine.QrTables;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/**
 * Functional verification of the in-house encoder component against the independent ZXing oracle.
 *
 * <p>The checks are deliberately layered: first the ISO/IEC 18004 tables are compared with the
 * tables published by ZXing, then real symbols are decoded, then every one of the 160
 * version/level combinations is exercised at its exact byte capacity, and finally the failure
 * paths are exercised so that a "green" report cannot be produced by silent no-ops.</p>
 */
final class EngineReferenceTest {

    private EngineReferenceTest() {
    }

    static void run(TestHarness harness, List<String> tableCrossCheckCsv, List<String> capacityCsv) {
        // ------------------------------------------------------------------ table cross-check
        int mismatches = 0;
        StringBuilder detail = new StringBuilder();
        for (int version = 1; version <= 40; version++) {
            for (QrErrorCorrection level : QrErrorCorrection.values()) {
                int[] zxing = ZxingVerify.zxingCodewordCounts(version, level);
                int eccPerBlock = QrTables.eccCodewordsPerBlock(level, version);
                int blockCount = QrTables.numErrorCorrectionBlocks(level, version);
                int dataCodewords = QrTables.numDataCodewords(version, level);
                int totalCodewords = QrTables.totalCodewords(version);
                boolean same = zxing[0] == eccPerBlock && zxing[1] == blockCount
                        && zxing[2] == dataCodewords && zxing[3] == totalCodewords;
                if (!same) {
                    mismatches++;
                    detail.append(String.format("v%d-%s ", version, level.getLabel()));
                }
                tableCrossCheckCsv.add(String.format("%d,%s,%d,%d,%d,%d,%d,%d,%d,%d,%s",
                        version, level.getLabel(), zxing[0], eccPerBlock, zxing[1], blockCount,
                        zxing[2], dataCodewords, zxing[3], totalCodewords,
                        same ? "match" : "MISMATCH"));
            }
        }
        harness.check("engine", "ISO 18004 tables match ZXing for 40 versions x 4 levels",
                mismatches == 0, mismatches + " mismatching combinations " + detail);

        // ------------------------------------------------------------------ payload round trips
        String[] payloads = {
            "A",
            "HELLO WORLD",
            "1234567890",
            "12345678901234567890123456789012345678901234567890123456789012345678901234567890",
            "https://www.ku.ac.ke/sco310/component-programming",
            "Kenyatta University - SCO310 Component Programming 2025/2026",
            "Mixed case, punctuation: .,;:!? and symbols /+-%*",
            "\u00e9\u00e8\u00ea caf\u00e9 na\u00efve r\u00e9sum\u00e9",
            "\u4f60\u597d \u4e16\u754c - Chinese text payload",
            "\ud83d\ude80 Emoji payload with a rocket",
            "tel:+254700000000",
            "WIFI:T:WPA;S:SCO310-Lab;P:component-programming;;",
            "BEGIN:VCARD\nVERSION:3.0\nFN:Component Programming\nEND:VCARD",
            "line1\nline2\tTAB\rcarriage",
            "0123456789abcdefghijklmnopqrstuvwxyz",
        };
```

Screenshot 10. The bean feature matrix across four components in one view.

feature discovery for four components: properties, events, methods

```
student@ku:~/qr-component-studio$ python3 tools/bean_feature_matrix.py
============================================================
BEAN INSPECTION: ke.ku.sco310.qr.components.PureJavaQrEncoderBean
============================================================

[1] Required JavaBeans characteristics
  public class                    : yes
  no-argument constructor         : yes
  implements Serializable         : yes
  superclass                      : java.lang.Object
  directly implemented interfaces  : ke.ku.sco310.qr.api.QrCodeEncoder, java.io.Serializable,
    ke.ku.sco310.qr.api.BoundPropertySource
  explicit BeanInfo               : none (reflection only)
  display name                    : PureJavaQrEncoderBean
  short description               : PureJavaQrEncoderBean

[2] Properties discovered through the get/set design patterns
PROPERTY                TYPE                          READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
------------------------------------------------------------------------------------------------------------------------
    --------------
boostErrorCorrection    boolean                       yes    yes    -         normal    -
componentId             String                        yes    -      -         normal    -
defaultErrorCorrection  QrErrorCorrection             yes    yes    -         normal    -
encoderInfo             EncoderInfo                   yes    -      -         normal    -
forcedMaskPattern       int                           yes    yes    -         normal    -
maximumVersion          int                           yes    yes    -         normal    -
minimumVersion          int                           yes    yes    -         normal    -
  properties found                : 7

[3] Event sets (discovered from add/remove listener methods)
  propertyChange                  listener=PropertyChangeListener                   add=addPropertyChangeListener
  vetoableChange                  listener=VetoableChangeListener                   add=addVetoableChangeListener
  event sets                      : 2

[4] Public methods that are neither accessors nor listener plumbing
  encode                       returns QrCode
  encode                       returns QrCode
  wait                         returns void
  notifyAll                    returns void
  notify                       returns void
  wait                         returns void
  hashCode                     returns int
  wait                         returns void
  equals                       returns boolean
  toString                     returns String

Summary: 7 properties, 2 event sets, 27 methods


============================================================
BEAN INSPECTION: ke.ku.sco310.qr.render.SvgRendererBean
============================================================

[1] Required JavaBeans characteristics
  public class                    : yes
  no-argument constructor         : yes
  implements Serializable         : yes
  superclass                      : java.lang.Object
  directly implemented interfaces  : ke.ku.sco310.qr.api.QrCodeRenderer, java.io.Serializable,
    ke.ku.sco310.qr.api.BoundPropertySource
  explicit BeanInfo               : none (reflection only)
  display name                    : SvgRendererBean
  short description               : SvgRendererBean

[2] Properties discovered through the get/set design patterns
PROPERTY                TYPE                          READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
------------------------------------------------------------------------------------------------------------------------
    --------------
backgroundColor         String                        yes    yes    -         normal    -
fileExtension           String                        yes    -      -         normal    -
foregroundColor         String                        yes    yes    -         normal    -
formatName              String                        yes    -      -         normal    -
mergeHorizontalRuns     boolean                       yes    yes    -         normal    -
mimeType                String                        yes    -      -         normal    -
moduleSize              int                           yes    yes    -         normal    -
physicalWidthMillimetres double                       yes    yes    -         normal    -
quietZone               int                           yes    yes    -         normal    -
rendererId              String                        yes    -      -         normal    -
  properties found                : 10
```

```
[3] Event sets (discovered from add/remove listener methods)
  propertyChange                  listener=PropertyChangeListener                    add=addPropertyChangeListener
  event sets                      : 1

[4] Public methods that are neither accessors nor listener plumbing
  describeConfiguration        returns String
  wait                         returns void
  notifyAll                    returns void
  notify                       returns void
  wait                         returns void
  hashCode                     returns int
  countRectangles              returns int
  render                       returns RenderedArtifact
  wait                         returns void
  equals                       returns boolean
  toString                     returns String

Summary: 10 properties, 1 event sets, 30 methods


============================================================
BEAN INSPECTION: ke.ku.sco310.qr.store.FileSystemRepositoryBean
============================================================

[1] Required JavaBeans characteristics
  public class                    : yes
  no-argument constructor         : yes
  implements Serializable         : yes
  superclass                      : java.lang.Object
  directly implemented interfaces  : ke.ku.sco310.qr.api.QrCodeRepository, java.io.Serializable,
    ke.ku.sco310.qr.api.BoundPropertySource
  explicit BeanInfo               : none (reflection only)
  display name                    : FileSystemRepositoryBean
  short description               : FileSystemRepositoryBean

[2] Properties discovered through the get/set design patterns
PROPERTY                TYPE                           READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
------------------------------------------------------------------------------------------------------------------------------------------------
    --------------
componentId             String                         yes    -      -         normal    -
historyCount            int                            yes    -      -         normal    -
historyFileName         String                         yes    yes    -         normal    -
historySnapshot         List                           yes    -      -         normal    -
rootDirectory           File                           yes    yes    -         normal    -
storedArtifactCount     int                            yes    -      -         normal    -
  properties found              : 6

[3] Event sets (discovered from add/remove listener methods)
  propertyChange                  listener=PropertyChangeListener                    add=addPropertyChangeListener
  event sets                      : 1

[4] Public methods that are neither accessors nor listener plumbing
  storeArtifact               returns File
  appendHistory               returns void
  describeConfiguration       returns String
  wait                        returns void
  notifyAll                   returns void
  notify                      returns void
  wait                        returns void
  clearHistory                returns void
  hashCode                    returns int
  loadHistory                 returns List
  wait                        returns void
  equals                      returns boolean
  toString                    returns String

Summary: 6 properties, 1 event sets, 24 methods


============================================================
BEAN INSPECTION: ke.ku.sco310.qr.log.AuditLogListenerBean
============================================================

[1] Required JavaBeans characteristics
  public class                    : yes
  no-argument constructor         : yes
  implements Serializable         : yes
  superclass                      : java.lang.Object
  directly implemented interfaces  : ke.ku.sco310.qr.event.QrCodeGenerationListener, java.io.Serializable,
    ke.ku.sco310.qr.api.BoundPropertySource
  explicit BeanInfo               : none (reflection only)
  display name                    : AuditLogListenerBean
```

```
  display name                  : AuditLogListenerBean
  short description             : AuditLogListenerBean

[2] Properties discovered through the get/set design patterns
PROPERTY                TYPE                          READ   WRITE  INDEXED   CATEGORY  PROPERTY EDITOR
------------------------------------------------------------------------------------------------------------------------
    --------------
componentId             String                        yes    -      -         normal    -
echoToConsole           boolean                       yes    yes    -         normal    -
logFile                 File                          yes    yes    -         normal    -
processedEventCount     int                           yes    -      -         normal    -
recentLines             List                          yes    -      -         normal    -
verbose                 boolean                       yes    yes    -         normal    -
  properties found              : 6

[3] Event sets (discovered from add/remove listener methods)
  propertyChange                listener=PropertyChangeListener                   add=addPropertyChangeListener
  event sets                    : 1

[4] Public methods that are neither accessors nor listener plumbing
  wait                      returns void
  notifyAll                 returns void
  qrCodeGenerated           returns void
  notify                    returns void
  wait                      returns void
  hashCode                  returns int
  wait                      returns void
  equals                    returns boolean
  toString                  returns String

Summary: 6 properties, 1 event sets, 21 methods
```

Screenshot 11. The artifact returned by the HTTP service for a PNG request.

